Appunti di Informatica Teorica

13 maggio 2026

Indice

1 Introduzione **3**

2 Teoria dellla computabilità **4**
2.1 Fondamenti della Calcolabilità . . . 4
2.1.1 Enumerabilità, computabilità e decidibilità . . . 4
2.2 Funzioni Ricorsive . . . 7
2.2.1 Funzioni ricorsive primitive . . . 7
2.2.2 Relazioni ricorsive primitive . . . 12
2.2.3 Funzione di Ackermann e funzioni μ-ricorsive . . . 15
2.2.4 Funzioni μ-ricorsive . . . 17
2.3 Macchine di Turing . . . 18
2.3.1 Macchine di Turing come accettatori di linguaggi . . . 22
2.3.2 Varianti di Macchine di Turing . . . 24
2.4 MDT Non Deterministica . . . 27
2.5 Funzioni Parziali Computabili . . . 28
2.6 Enumerazione e Problema dell'Arresto . . . 29
2.6.1 Codifica ed Enumerazione delle MDT . . . 29
2.6.2 L'Insieme K e l'Indecidibilità . . . 30
2.7 Macchina Universale e Riducibilità . . . 32
2.7.1 Problema del Nastro Vuoto (BTP) . . . 33
2.7.2 Proprietà dei Linguaggi Semidecidibili . . . 34
2.7.3 Il Teorema di Rice . . . 35
2.7.4 Equivalenza tra Modelli di Calcolo . . . 37

3 Teoria della Complessità **38**
3.1 Notazioni Asintotiche . . . 38
3.2 Complessità in tempo su MDT Non Deterministiche . . . 40
3.3 Le Classi P e NP . . . 42
3.3.1 Problema del circuito hamiltoniano (HAM) . . . 42
3.3.2 Riducibilità Polinomiale . . . 45
3.3.3 Classi NP-difficile e NP-completo . . . 45
3.3.4 Relazione tra P, NP e NPC . . . 46
3.4 Il Problema SAT (Satisfiability) . . . 46
3.4.1 Rappresentazione delle Istanze di un Problema . . . 46
3.4.2 Polinomi Booleani . . . 46
3.4.3 Definizione del Problema SAT . . . 47
3.5 Teorema di Cook: SAT è NP-difficile . . . 48
3.6 Proprietà della classe NP-Completo e NP-Difficile . . . 51
3.7 Il Problema 3-SAT . . . 52
3.8 Problema del Vertex Cover (VC) . . . 52
3.9 Problema della CLIQUE . . . 53
3.10 Problema del Circuito Hamiltoniano (HAM) . . . 55
3.10.1 Costruzione del Grafo HAM Completo (Esempio) . . . 55
3.11 Problema dell'Insieme Indipendente (IS) . . . 57
3.12 Problema 2-SAT . . . 57

3.13 Linguaggi NP-Intermedi (NPI) 59
3.14 La classe co-NP 59
3.14.1 Proprietà di Chiusura dei Linguaggi co-NP 60
3.15 Test di Primalità 61
3.16 Esercizio: Macchina di Turing e Transizioni 63

1 Introduzione

L'Informatica Teorica si divide in due macro-aree fondamentali:

- **Teoria della computabilità**: cerca modelli formali che si avvicinino al concetto logico e matematico di ALGORITMO.
- **Teoria della complessità computazionale**: si occupa di determinare la quantità minima di risorse (principalmente spazio e tempo) necessarie ad un algoritmo per risolvere un dato problema.

2 Teoria dellla computabilità

2.1 Fondamenti della Calcolabilità

2.1.1 Enumerabilità, computabilità e decidibilità

Cos'è un algoritmo? È una NOZIONE PRIMITIVA $\longrightarrow$ non ha una definizione formale, ma lo abbiamo nella nostra testa e sappiamo riconoscerlo.

Definizioni formali
Per prima cosa ci mettiamo in un <u>contesto formale</u>:

- Σ : alfabeto
- Σ^* : stringhe finite su Σ
- $\mathcal{L} \subseteq \Sigma^*$: linguaggio

Definizione: $\mathcal{L} \subseteq \Sigma^*$ si dice **DECIDIBILE** quando $\exists\ \mathcal{M}$ algoritmo che, data $w \in \Sigma^*$, mi dice in tempo finito se $w \in \mathcal{L}$ oppure $w \notin \mathcal{L}$.

Definizione: $\mathcal{L} \subseteq \Sigma^*$ si dice **ENUMERABILE** quando $\exists\ \mathcal{M}$ algoritmo che, fatto partire su input vuoto, <u>produce la lista delle stringhe di $\mathcal{L}$</u> (ciascuna stringa in tempo finito).

Definizione: Siano Σ_1, Σ_2 alfabeti, $\mathcal{L}_1 \subseteq \Sigma_1^*$, $\mathcal{L}_2 \subseteq \Sigma_2^*$.
$f : \mathcal{L}_1 \to \mathcal{L}_2$ si dice **COMPUTABILE** quando $\exists\ \mathcal{M}$ algoritmo che, fatto partire su input $w \in \mathcal{L}_1$, restituisce $f(w) \in \mathcal{L}_2$ in tempo finito.

Proposizione: Sia Σ un alfabeto.

$$\mathcal{L} \text{ DECIDIBILE} \implies \mathcal{L} \text{ ENUMERABILE}$$

Dimostrazione:
Sia $\mathcal{M}$ un algoritmo di decisione per $\mathcal{L}$. Descrivo un algoritmo di enumerazione per $\mathcal{L}$: $\forall$ stringa $w \in \Sigma^*$, eseguo $\mathcal{M}$ su w:

- Se $w \in \mathcal{L}$, aggiungo w alla lista.
- Se $w \notin \mathcal{L}$, non l'aggiungo.

Osservazione: Σ numerabile $\implies$ Σ^* enumerabile.
Sia $\Sigma = \{a_1, a_2, \dots, a_n, \dots\}$.
Se cerchiamo di enumerare le stringhe di lunghezza 2 su Σ tramite un PROCEDIMENTO DIAGONALE DI CANTOR:

$$\begin{array}{c|ccccccc}
 & a_1 & & a_2 & & a_3 & & \dots \\
\hline
a_1 & a_1a_1 & \rightarrow & a_1a_2 & & a_1a_3 & \rightarrow & \dots \\
 & & \swarrow & & \nearrow & & \swarrow & \\
a_2 & a_2a_1 & & a_2a_2 & & a_2a_3 & & \dots \\
 & \downarrow & \nearrow & & \swarrow & & & \\
a_3 & a_3a_1 & & a_3a_2 & & \dots & & \\
 & & \swarrow & & & & & \\
\vdots & \vdots & & \vdots & & & & \ddots
\end{array}$$

Usando il procedimento diagonale è possibile enumerare le parole e poi trovare un algoritmo di enumerazione per le stringhe.

Attenzione: $\mathcal{L}$ decidibile $\not\Leftarrow$ $\mathcal{L}$ enumerabile (IN GENERALE).
MA vale la seguente...

Proposizione:

$$\mathcal{L} \text{ DECIDIBILE} \iff \mathcal{L} \text{ ENUMERABILE e } \mathcal{L}^c \text{ ENUMERABILE} \quad (\text{dove } \mathcal{L}^c = \Sigma^* \setminus \mathcal{L})$$

Dimostrazione:

$\Longrightarrow$) $\mathcal{L}$ decidibile $\Longrightarrow$ $\mathcal{L}$ enumerabile (già dimostrato, OK).
$\mathcal{L}$ decidibile $\Longrightarrow$ $\mathcal{L}^c$ enumerabile: analogo.

$\Longleftarrow$) Sia $\mathcal{M}$ algoritmo di enumerazione per $\mathcal{L}$: $w_1, w_2, w_3, \ldots, w_n, \ldots$
Sia $\mathcal{M}^c$ algoritmo di enumerazione per $\mathcal{L}^c$: $v_1, v_2, v_3, \ldots, v_n, \ldots$
Costruisco un algoritmo di decisione per $\mathcal{L}$:
Su input $w \in \Sigma^*$, eseguo $\mathcal{M}$ e $\mathcal{M}^c$ in maniera ALTERNATA. Ogni volta che viene prodotta una stringa X, la confronto con w:

- Se $X \neq w$, proseguo.
- Se $X = w$ e X è prodotta da $\mathcal{M}$, accetto w.
- Se $X = w$ e X è prodotta da $\mathcal{M}^c$, rifiuto w.

Definizione: $\mathcal{L} \subseteq \Sigma^*$ si dice **SEMIDECIDIBILE** quando $\exists$ $\mathcal{M}$ algoritmo che, fatto partire su input w:

- Se $w \in \mathcal{L}$, termina e accetta w.
- Se $w \notin \mathcal{L}$, può non terminare (va in loop).

Osservazione: $\mathcal{L}$ semidecidibile $\iff$ $\exists$ $\mathcal{M}$ algoritmo che, su input w:

- Se $w \in \mathcal{L}$, termina e accetta w.
- Se $w \notin \mathcal{L}$, non termina.

Nota pratica: Questa equivalenza è utilissima nelle dimostrazioni perché ci permette di assumere che esista un solo modo in cui la macchina fallisce (il loop infinito), togliendoci il dubbio che possa fermarsi per rifiutare.

Proposizione:

$$\mathcal{L} \text{ ENUMERABILE} \iff \mathcal{L} \text{ SEMIDECIDIBILE}$$

Dimostrazione:

$\Longrightarrow$) Sia $\mathcal{M}$ algoritmo di enumerazione per $\mathcal{L}$: $w_1, w_2, w_3, \ldots$
Costruisco un algoritmo di semidecisione per $\mathcal{L}$:
Data $w \in \Sigma^*$, confronto w con le stringhe prodotte via via da $\mathcal{M}$. Se w è tra queste, termino e accetto w.

$\Longleftarrow$) Sia $\mathcal{M}$ algoritmo di semidecisione per $\mathcal{L}$ (che non termina su tutte le stringhe del complemento).
Eseguo i passi di computazione intrecciandoli (dovetailing):

$$\begin{array}{c|cccccccc}
w_1: & x & & x & & x & \ldots & x \\
 & & \swarrow & & \nearrow & & \swarrow & \\
w_2: & x & & x & & x & \ldots & \circ \\
 & \downarrow & \nearrow & & \swarrow & & & \\
w_3: & x & & x & & \ldots & & \\
 & & \swarrow & & & & & \\
w_4: & x & & \ldots & & & & \\
\vdots & \vdots & & & & & &
\end{array}$$

(Eseguo 1 passo su w_1; poi 1 passo su w_1 e 1 su w_2; poi 1 passo su w_1, w_2 e w_3, ecc. Quando una computazione termina e accetta (es. in w_2), metto la stringa corrispondente nella lista).

Definizione: $\mathcal{L} \subseteq \Sigma^*$ si dice avere **FUNZIONE CARATTERISTICA** $\chi_{\mathcal{L}}$ se la funzione è così definita:

$$\chi_{\mathcal{L}} : \Sigma^* \to \{0,1\} \quad : \quad w \mapsto \begin{cases} 0, & w \notin \mathcal{L} \\ 1, & w \in \mathcal{L} \end{cases}$$

Proposizione:

$$\mathcal{L} \text{ DECIDIBILE} \iff \chi_{\mathcal{L}} \text{ COMPUTABILE}$$

Dimostrazione:

$\Longrightarrow$) Sia $\mathcal{M}$ algoritmo di decisione per $\mathcal{L}$. Costruisco un algoritmo di calcolo per $\chi_{\mathcal{L}}$:
Data $w \in \Sigma^*$, eseguo $\mathcal{M}$ su w:

- Se $w \in \mathcal{L}$, scrivo 1.
- Se $w \notin \mathcal{L}$, scrivo 0.

$\Longleftarrow$) Sia $\mathcal{M}$ algoritmo di calcolo per $\chi_{\mathcal{L}}$. Costruisco un algoritmo di decisione per $\mathcal{L}$:
Su input w, calcolo $\chi_{\mathcal{L}}(w)$ (usando $\mathcal{M}$):

- Se $\chi_{\mathcal{L}}(w) = 1$, $w \in \mathcal{L}$.
- Se $\chi_{\mathcal{L}}(w) = 0$, $w \notin \mathcal{L}$.

Proposizione: Siano $\mathcal{L}_1, \mathcal{L}_2 \subseteq \Sigma^*$ DECIDIBILI. Allora:

1. $\mathcal{L}_1 \cup \mathcal{L}_2$ è decidibile
2. $\mathcal{L}_1 \cap \mathcal{L}_2$ è decidibile
3. $\mathcal{L}_1^c$ è decidibile

Dimostrazione:

1. **Algoritmo di decisione per $\mathcal{L}_1 \cup \mathcal{L}_2$:**
   Su input w: Eseguo $\mathcal{M}_1$. Se $w \in \mathcal{L}_1$ termino e accetto.
   Se $w \notin \mathcal{L}_1$, eseguo $\mathcal{M}_2$:
   - Se $w \in \mathcal{L}_2$, termino e accetto.
   - Altrimenti rifiuto.
2. **Algoritmo di decisione per $\mathcal{L}_1 \cap \mathcal{L}_2$:**
   Su input w: Eseguo $\mathcal{M}_1$:
   - Se $w \notin \mathcal{L}_1$, rifiuto.
   - Se $w \in \mathcal{L}_1$, osservo $\mathcal{M}_2$:
     - Se $w \notin \mathcal{L}_2$, rifiuto.
     - Altrimenti accetto.
3. **Algoritmo di decisione per $\mathcal{L}_1^c$:**
   Su input w, eseguo $\mathcal{M}_1$ e SCAMBIO LE SUE RISPOSTE (cioè l'algoritmo di decisione inverte l'output).

2.2 Funzioni Ricorsive

2.2.1 Funzioni ricorsive primitive

Ci concentriamo su funzioni del tipo:

$$f : \mathbb{N}^k \to \mathbb{N}$$
$$(x_1, \dots, x_k) \in \mathbb{N}^k$$
$$f(x_1, \dots, x_k)$$

Indicheremo il vettore di input come $\vec{x} = (x_1, \dots, x_k)$.

FUNZIONI INIZIALI

1. **Costanti k-arie di valore 0**:
$$C_0^{(k)} : \mathbb{N}^k \to \mathbb{N}$$
$$C_0^{(k)}(\vec{x}) = 0$$

2. **Funzione successore**:
$$S : \mathbb{N} \to \mathbb{N}$$
$$S(x) = x + 1$$

3. **Proiezioni j-esime k-arie**:
$$\mathcal{E}_j^{(k)} : \mathbb{N}^k \to \mathbb{N}$$
$$\mathcal{E}_j^{(k)}(\vec{x}) = x_j \quad \text{(estrae l'argomento j-esimo)}$$

Definizione: Sia A un insieme, Σ un alfabeto.
Una **Codifica** è una funzione $p : A \to \Sigma^*$ (da A ottengo una stringa).

Definizione: Una **Rappresentazione** (Buona codifica) di A in Σ si ha quando:

1. p è **computabile** ($\exists$ algoritmo che per $\forall\ a \in A$ mi calcola $p(a)$).
2. p è **iniettiva** (la codifica è unica).
3. $p(A)$ è **decidibile** e p è **effettivamente invertibile**, cioè: $\exists$ un algoritmo che, $\forall\ w \in p(A)$, mi restituisce $a \in A$ tale che $p(a) = w$ (Posso ricostruire senza ambiguità l'elemento di A dalla codifica).

Proposizione: Sia Σ un alfabeto. Allora $\exists\ p$ rappresentazione in $\{0,1\}^*$.
$p : \Sigma^* \to \{0,1\}^*$ (Posso codificare in binario).
Dimostrazione:
Sia $\Sigma = \{a_1, a_2, \dots, a_n, \dots\}$.
$p : \Sigma^* \to \{0,1\}^*$
$p(a_{i_1} a_{i_2} \dots a_{i_n}) = 01\dots101\dots1\dots01\dots1$ (dove i blocchi di 1 dipendono dagli indici i_k).
p è una rappresentazione di Σ^* in $\{0,1\}^*$.

Codifica unaria per numeri naturali:
$\Sigma = \{1\}$
$0 \mapsto 1$
$1 \mapsto 11$
$2 \mapsto 111$
$n \mapsto \underbrace{11\dots1}_{n+1 \text{ volte}}$

Osservazione: Le funzioni iniziali sono computabili.
Esempio: $S(\underbrace{11\dots1}_{n+1}) = \underbrace{1\dots1}_{n+2}$

Definizione: Composizione generalizzata
Date $g_1 \dots g_n : \mathbb{N}^k \to \mathbb{N}$ e $f : \mathbb{N}^n \to \mathbb{N}$.
La **composizione generalizzata** (o sostituzione) di f con $(g_1, \dots, g_n)$ è la funzione:

$$f \circ (g_1, \dots, g_n) : \mathbb{N}^k \to \mathbb{N}$$

$$(f \circ (g_1, \dots, g_n))(\vec{x}) = f(g_1(\vec{x}), \dots, g_n(\vec{x}))$$

Proposizione: $f, g_1, \dots, g_n$ computabili $\implies f \circ (g_1, \dots, g_n)$ computabile.
Dimostrazione:
Siano $F, G_1, \dots, G_n$ algoritmo di calcolo per $f, g_1, \dots, g_n$.
Algoritmo di calcolo per $f \circ (g_1, \dots, g_n)$:
Data $\vec{x} \in \mathbb{N}^k$:

- Uso $G_1, \dots, G_n$ per calcolare $g_1(\vec{x}), \dots, g_n(\vec{x})$.
- Uso F per calcolare $f(g_1(\vec{x}), \dots, g_n(\vec{x}))$.

Definizione: Ricorsione Primitiva (RP)
Date $g : \mathbb{N}^k \to \mathbb{N}$, $\varphi : \mathbb{N}^{k+2} \to \mathbb{N}$.
Diciamo che $f : \mathbb{N}^{k+1} \to \mathbb{N}$ è definita per **ricorsione primitiva** (RP) da g e φ quando:

- $f(\vec{x}, 0) = g(\vec{x})$ *(caso base)*
- $f(\vec{x}, y+1) = \varphi(\vec{x}, y, f(\vec{x}, y))$ *(passo induttivo)*

Proposizione: g, φ computabili $\implies f$ definita per RP da g, φ è computabile.
Dimostrazione:
Descrivo un algoritmo per calcolare f, ragionando per induzione sull'ultimo parametro di f. In pratica facciamo vedere che fissato $\vec{x} \in \mathbb{N}^k$, per ogni $y \in \mathbb{N}$, so calcolare $f(\vec{x}, y)$:

- (Base $y = 0$): $f(\vec{x}, 0) = g(\vec{x})$, e g è computabile.
- (Iterazione): Supponiamo di saper calcolare $f(\vec{x}, y)$.
  $f(\vec{x}, y+1) = \varphi(\vec{x}, y, f(\vec{x}, y))$. Essendo φ computabile (per hp) e $f(\vec{x}, y)$ noto, anche il calcolo al passo successivo restituisce un risultato in tempo finito.

Definizione: Derivazione Ricorsiva Primitiva (drp)
È una sequenza di funzioni $f_1, f_2, \dots, f_n$ t.c. $\forall\ i = 1, \dots, n$:

- f_i è **funzione iniziale**, oppure
- f_i è **ottenuta da** f_j, f_k **per RP** con $j, k < i$, oppure
- f_i è **ottenuta per composizione generalizzata** da funzioni precedenti

Definizione: Funzione Ricorsiva Primitiva (rp)
Una **funzione ricorsiva primitiva (rp)** è una funzione che compare in coda ad una drp (ovvero è l'ultima funzione di una drp).

Osservazione:

1. Ogni prefisso di una drp è una drp.
2. Ogni funzione che compare in una drp è rp.

Proposizione: Ogni funzione rp è computabile.
Dimostrazione:
Sia f rp. Dimostro per **induzione sulla lunghezza n di una drp** che termina con f:

- **Caso base** ($n = 1$): $f \implies f$ è funzione iniziale, dunque è computabile.
- **Passo induttivo**: Supponiamo che tutte le funzione che compaiono in una drp di lunghezza $< n$ siano computabili.
  Sia $f_1, \ldots, f_n = f$.
  f deriva da funzioni precedenti $f_j, \ldots, f_k$ le quali sono computabili per Hp induttiva. Siccome le operazioni di composizione e RP mantengono la computabilità (come dimostrato in precedenza), anche f è computabile.

Proposizione:

1. $g : \mathbb{N}^k \to \mathbb{N}$, $\varphi : \mathbb{N}^{k+2} \to \mathbb{N}$ rp $\implies f$ ottenuta per RP da g e φ è rp.
2. $\varphi : \mathbb{N}^n \to \mathbb{N}$, $g_1, \ldots, g_n : \mathbb{N}^k \to \mathbb{N}$ rp $\implies f \circ (g_1, \ldots, g_n)$ rp.

Dimostrazione:

1. Siano $g_1, \ldots, g_s, g$ una drp per g, e $\varphi_1, \ldots, \varphi_r, \varphi$ una drp per φ.
   La sequenza $g_1, \ldots, g_s, g, \varphi_1, \ldots, \varphi_r, \varphi, f$ è essa stessa una drp (poiché per ipotesi tutte le precedenti sono drp e f è ottenuta da esse). Di conseguenza, f è rp perché compare in una drp.

ESEMPI DI FUNZIONI RICORSIVE PRIMITIVE

1) Costante m-esima k-aria: $C_m^{(k)} : \mathbb{N}^k \to \mathbb{N}$, dove $C_m^{(k)}(x_1, \ldots, x_k) = m \quad (m \in \mathbb{N})$, con k che indica il numero di argomenti (es. $C_0^{(2)}(x, y) = 0$) e m che indica il valore della costante (es. $C_3^{(2)}(x, y) = 3$). Dimostriamo per induzione su $m \in \mathbb{N}$ che è rp:

- Base: $C_0^{(k)}$ è rp (perché è una **funzione iniziale**).
- Supponiamo che $C_m^{(k)}$ sia rp.
  $C_{m+1}^{(k)}(\vec{x}) = m + 1 = (S \circ C_m^{(k)})(\vec{x})$
  Essendo la composizione tra S (funzione successore, che è iniziale) e $C_m^{(k)}$ (rp per ipotesi induttiva), allora $C_{m+1}^{(k)}$ è rp.

2) Funzione Predecessore: $v : \mathbb{N} \to \mathbb{N}$

$$v(x) = \begin{cases} x - 1, & x > 0 \\ 0, & x = 0 \end{cases}$$

Costruiamo la ricorsione primitiva:

$$\begin{cases} v(0) = 0 \quad \text{(che corrisponde a } C_0^{(0)} \text{ che è rp)} \\ v(x+1) = \left(\mathcal{E}_1^{(2)}\right)(x, v(x)) = x \quad \text{(rp perché } \mathcal{E}_1^{(2)} \text{ è funz. iniziale)} \end{cases}$$

$\implies v(x)$ è rp.

3) Somma: $s : \mathbb{N}^2 \to \mathbb{N}$
$s(x, y) = x + y$

$$\begin{cases} s(x,0) = x = \mathcal{E}_1^{(1)}(x) \quad \text{(rp)} \\ s(x, y+1) = x + y + 1 = \left(S \circ \mathcal{E}_3^{(3)}\right)(x, y, s(x,y)) \quad \text{(rp)} \end{cases}$$

$\implies$ s è rp.

4) Prodotto: $p : \mathbb{N}^2 \to \mathbb{N}$
$p(x, y) = x \cdot y$

$$\begin{cases} p(x,0) = 0 = \left(C_0^{(1)}\right)(x) \quad \text{(rp)} \\ p(x, y+1) = xy + x = \left(s \circ (\mathcal{E}_3^{(3)}, \mathcal{E}_1^{(3)})\right)(x, y, p(x,y)) \quad \text{(rp)} \end{cases}$$

$\implies$ p è rp.

5) Esponenziazione: $\pi : \mathbb{N}^2 \to \mathbb{N}$
$\pi(x, y) = x^y$

$$\begin{cases} \pi(x,0) = x^0 = 1 = C_1^{(1)}(x) \quad \text{(rp)} \\ \pi(x, y+1) = x^y \cdot x = \left(p \circ (\mathcal{E}_3^{(3)}, \mathcal{E}_1^{(3)})\right)(x, y, \pi(x,y)) \quad \text{(rp)} \end{cases}$$

6) Fattoriale: $f : \mathbb{N} \to \mathbb{N}$
$f(x) = x!$

$$\begin{cases} f(0) = 1 \quad \text{(rp)} \\ f(x+1) = (x+1)! = (x+1) \cdot x! = \left(p \circ (S \circ \mathcal{E}_1^{(2)}, \mathcal{E}_2^{(2)})\right)(x, f(x)) \end{cases}$$

7) Differenza Troncata: $\dot{-} : \mathbb{N}^2 \to \mathbb{N}$

$$x \dot{-} y = \begin{cases} x - y, & x \geq y \\ 0, & x < y \end{cases}$$

$$\begin{cases} x \dot{-} 0 = x = \mathcal{E}_1^{(1)}(x) \quad \text{(rp)} \\ x \dot{-} (y+1) = x - y - 1 = \left(v \circ \mathcal{E}_3^{(3)}\right)(x, y, x \dot{-} y) \quad \text{(rp)} \end{cases}$$

8) Distanza: $d : \mathbb{N}^2 \to \mathbb{N}$
$d(x, y) = |x - y|$

$$d(x,y) = (x \dot{-} y) + (y \dot{-} x)$$

(Questo copre entrambi i casi in cui $x \geq y$ o $y > x$)

$$= \left(s \circ (\dot{-} \circ (\mathcal{E}_1^{(2)}, \mathcal{E}_2^{(2)}), \dot{-} \circ (\mathcal{E}_2^{(2)}, \mathcal{E}_1^{(2)}))\right)(x, y)$$

$\implies$ d è rp.

9) Segno: $sg : \mathbb{N} \to \mathbb{N}$

$$sg(x) = \begin{cases} 1, & x > 0 \\ 0, & x = 0 \end{cases}$$

$$\begin{cases} sg(0) = 0 \\ sg(x+1) = 1 = C_1^{(2)}(x, sg(x)) \end{cases}$$

10) Segno Opposto: $\overline{sg} : \mathbb{N} \to \mathbb{N}$

$$\overline{sg}(x) = \begin{cases} 0, & x > 0 \\ 1, & x = 0 \end{cases}$$

Dimostrazione: Possiamo dimostrare che è rp per ricorsione primitiva:

$$\begin{cases} \overline{sg}(0) = 1 = C_1^{(0)} \quad \text{(rp)} \\ \overline{sg}(x+1) = 0 = C_0^{(2)}(x, \overline{sg}(x)) \quad \text{(rp, con composizione di proiezioni se necessario)} \end{cases}$$

$\implies \overline{sg}$ è rp.

Metodo alternativo (per composizione): Possiamo sfruttare la differenza troncata. Sottrarre x a 1 ci restituisce 1 solo se $x = 0$, altrimenti ci restituisce 0:

$$\overline{sg}(x) = 1 \dot{-} x = \left(\dot{-} \circ (C_1^{(1)}, \mathcal{E}_1^{(1)})\right)(x)$$

Essendo ottenuta per composizione di funzioni rp, è rp.

11) Delta di Kronecker: $\delta(x, y)$

$$\delta(x, y) = \begin{cases} 1, & x = y \\ 0, & x \neq y \end{cases}$$

Dimostrazione: Sappiamo che la funzione distanza $d(x, y) = |x - y|$ (che abbiamo già dimostrato essere rp) vale 0 se e solo se $x = y$, ed è strettamente maggiore di 0 altrimenti. Se applichiamo la funzione *Segno Opposto* al risultato della distanza, otteniamo esattamente il comportamento del Delta di Kronecker:

$$\delta(x, y) = \overline{sg}(d(x, y)) = (\overline{sg} \circ d)(x, y)$$

Essendo il Delta di Kronecker ottenuto per composizione di due funzioni rp ($\overline{sg}$ e d), è esso stesso una funzione rp.

Definizione: Operatore di Somma Limitata
Sia $f : \mathbb{N}^{k+1} \to \mathbb{N}$. Definiamo $g : \mathbb{N}^{k+1} \to \mathbb{N}$ ponendo:

$$g(\vec{x}, z) = \sum_{y=0}^{z} f(\vec{x}, y)$$

Diciamo che g è ottenuta per **somma limitata** da f. La funzione che, data f, costruisce g si chiama operatore di somma limitata.

Proposizione: f rp $\implies g$ ottenuta per somma limitata da f è rp.
Dimostrazione:

$$\begin{cases} g(\vec{x}, 0) = \sum_{y=0}^{0} f(\vec{x}, y) = f(\vec{x}, 0) = \left(f \circ (\mathcal{E}_1^{(k)}, \mathcal{E}_2^{(k)}, \ldots, \mathcal{E}_k^{(k)}, C_0^{(k)})\right)(\vec{x}) \\ \\ g(\vec{x}, z+1) = \sum_{y=0}^{z+1} f(\vec{x}, y) = \sum_{y=0}^{z} f(\vec{x}, y) + f(\vec{x}, z+1) = \\ = \left(s \circ (\mathcal{E}_{k+2}^{(k+2)}, f \circ (\mathcal{E}_1^{(k+2)}, \ldots, \mathcal{E}_k^{(k+2)}, S \circ \mathcal{E}_{k+1}^{(k+2)}))\right)(\vec{x}, z, g(\vec{x}, z)) \end{cases}$$

Osservazione:
Data $f : \mathbb{N}^{k+2} \to \mathbb{N}$, possiamo avere $g : \mathbb{N}^{k+1} \to \mathbb{N}$ definita come:

$$g(\vec{x}, z) = \sum_{y=0}^{z} f(\vec{x}, y, z)$$

Definizione: Operatore di Prodotto Limitato
Sia $f : \mathbb{N}^{k+1} \to \mathbb{N}$. Definiamo $g : \mathbb{N}^{k+1} \to \mathbb{N}$ ponendo:

$$g(\vec{x}, z) = \prod_{y=0}^{z} f(\vec{x}, y)$$

Diciamo che g è ottenuta per **prodotto limitato** da f.

Proposizione: f rp $\implies$ g rp.
(La dimostrazione è la stessa della somma limitata, sostituendo l'operatore di somma s con quello di prodotto p).

Esercizio
Sia $g : \mathbb{N}^2 \to \mathbb{N}$ una funzione rp.

a) Sia $f_1(x, y, z_1, \ldots, z_n) = g(x, y)$. Dimostrare che f_1 è rp:

$$f_1(x, y, z_1, \ldots, z_n) = g(x, y) = \left(g \circ (\mathcal{E}_1^{(n+2)}, \mathcal{E}_2^{(n+2)})\right)(x, y, z_1, \ldots, z_n)$$

b) Sia $f_2(x, y) = g(y, x)$. Dimostrare che f_2 sia rp:

$$f_2(x, y) = \left(g \circ (\mathcal{E}_2^{(2)}, \mathcal{E}_1^{(2)})\right)(x, y)$$

c) Sia $f_3(x) = g(x, x)$. Dimostrare che f_3 sia rp:

$$f_3(x) = \left(g \circ (\mathcal{E}_1^{(1)}, \mathcal{E}_1^{(1)})\right)(x)$$

2.2.2 Relazioni ricorsive primitive

Definizione: Sia $R \subseteq \mathbb{N}^k$. Una funzione $f : \mathbb{N}^k \to \mathbb{N}$ si dice **BUONA per** R quando:

$$\forall\ \vec{x} \in \mathbb{N}^k, \quad f(\vec{x}) = 0 \iff \vec{x} \in R \text{ (La relazione è soddisfatta)}$$

Definizione: $R \subseteq \mathbb{N}^k$ (relazione k-aria) si dice **RICORSIVA PRIMITIVA (rp)** quando:

$$\exists\ f : \mathbb{N}^k \to \mathbb{N} \text{ rp t.c. } \forall\ \vec{x} \in \mathbb{N}^k, \quad f(\vec{x}) = 0 \iff \vec{x} \in R$$

(cioè esiste f rp che è buona per R).

Osservazione: $R \subseteq \mathbb{N}^k$ è rp $\iff$ χ_R (FUNZIONE CARATTERISTICA di R) è rp.

$$\chi_R : \mathbb{N}^k \to \mathbb{N}, \quad \chi_R(\vec{x}) = \begin{cases} 1, & \text{se } \vec{x} \in R \\ 0, & \text{se } \vec{x} \notin R \end{cases}$$

Dimostrazione:

$\implies$) R rp $\implies$ sia $f : \mathbb{N}^k \to \mathbb{N}$ rp BUONA PER R.

$$\chi_R(\vec{x}) = (\overline{sg} \circ f)(\vec{x})$$

Essendo $\overline{sg}$ rp e f rp (per ipotesi), la loro composizione χ_R è rp.

$\impliedby$) χ_R è rp. Osservo che $\overline{sg} \circ \chi_R$ è BUONA PER R.

Proposizione: $R \subseteq \mathbb{N}^k$ rp $\implies$ $R^c = \mathbb{N}^k \setminus R$ rp.
Dimostrazione:
R rp $\implies$ χ_R rp $\implies$ χ_R è funzione BUONA PER R^c ed è rp $\implies$ R^c rp.

Proposizione: $R, S \subseteq \mathbb{N}^k$ rp $\implies$ $R \cup S$ rp.
Dimostrazione:
χ_R, χ_S sono rp.

$$\chi_{R \cup S}(\vec{x}) = sg \circ (\chi_R + \chi_S)(\vec{x})$$

(La somma fa 1 se solo una è vera, o 2 se entrambe vere, poi il sg tronca a 1 se ≥ 2)

$$= (sg \circ (s \circ (\chi_R, \chi_S)))(\vec{x})$$

Essendo ottenuta per composizione di funzioni rp, $\implies R \cup S$ rp.

Proposizione: $R, S \subseteq \mathbb{N}^k$ rp $\implies R \cap S$ rp.
Dimostrazione:
χ_R, χ_S rp.

$$\chi_{R\cap S}(\vec{x}) = \chi_R(\vec{x}) \cdot \chi_S(\vec{x})$$
$$= (p \circ (\chi_R, \chi_S))(\vec{x})$$

Essendo ottenuta per composizione di funzioni rp (prodotto p), $\implies R \cap S$ rp.

Definizione: $f : \mathbb{N}^k \to \mathbb{N}$ si dice **DEFINITA PER CASI** a partire dalle funzioni $f_1, \ldots, f_n$ e dalle relazioni $R_1, \ldots, R_n$ quando:

$$f(\vec{x}) = \begin{cases} f_1(\vec{x}), & \vec{x} \in R_1 \\ f_2(\vec{x}), & \vec{x} \in R_2 \\ \vdots & \vdots \\ f_n(\vec{x}), & \vec{x} \in R_n \end{cases}$$

(Dove $R_1, \ldots, R_n$ è una partizione di $\mathbb{N}^k$). (questa definizione si collega all'idea di "if-then-else" in programmazione)

Proposizione: $f : \mathbb{N}^k \to \mathbb{N}$ definita per casi da $f_1, \ldots, f_n, R_1, \ldots, R_n$.
Se $f_1, \ldots, f_n$ rp e $R_1, \ldots, R_n$ rp $\implies f$ rp.
Dimostrazione:

$$f(\vec{x}) = f_1(\vec{x}) \cdot \chi_{R_1}(\vec{x}) + f_2(\vec{x}) \cdot \chi_{R_2}(\vec{x}) + \cdots + f_n(\vec{x}) \cdot \chi_{R_n}(\vec{x})$$

Poiché somma e prodotto di funzioni rp sono rp, f è rp.

Proposizione:

$$f(\vec{x}) = \begin{cases} f_1(\vec{x}), & \vec{x} \in R_1 \\ \vdots & \vdots \\ f_{n-1}(\vec{x}), & \vec{x} \in R_{n-1} \\ f_n(\vec{x}), & \text{ALTRIMENTI} \end{cases}$$

Se $f_1, \ldots, f_n$ rp e $R_1, \ldots, R_{n-1}$ rp $\implies f$ rp.
Dimostrazione:
Sappiamo che $R_n = \mathbb{N}^k \setminus (R_1 \cup \cdots \cup R_{n-1})$ (che corrisponde all' "altrimenti"). Essendo ottenuta tramite unione e complemento di insiemi rp, R_n è rp $\implies f$ rp.

Esempio
Sia $R \subseteq \mathbb{N}^2$ definita ponendo $(x, y) \in R$ quando $x \leq y$.

$$\chi_R(x, y) = \begin{cases} 1, & x \leq y \\ 0, & x > y \end{cases}$$

$$\chi_R(x, y) = sg(y \dot{-} x + \delta(x, y))$$
$$= \left(sg \circ (s \circ (\dot{-} \circ (\mathcal{E}_2^{(2)}, \mathcal{E}_1^{(2)}), \delta))\right)(x, y)$$

Definizione: Sia $R \subseteq \mathbb{N}^{k+1}$. $f : \mathbb{N}^k \to \mathbb{N}$ è definita per **MINIMIZZAZIONE** da R quando:

$$f(\vec{x}) = \begin{cases} \min\{z \in \mathbb{N} \mid (\vec{x}, z) \in R\}, & \text{SE TALE INSIEME NON È VUOTO} \\ 0, & \text{ALTRIMENTI} \end{cases}$$

Definizione: Sia $R \subseteq \mathbb{N}^{k+1}$. $f : \mathbb{N}^{k+1} \to \mathbb{N}$ è definita per **MINIMIZZAZIONE LIMITATA** da R quando:

$$f(\vec{x}, y) = \begin{cases} \min\{z \leq y \mid (\vec{x}, z) \in R\}, & \text{SE L'INSIEME NON È VUOTO} \\ 0, & \text{ALTRIMENTI} \end{cases}$$

Proposizione: R decidibile $\implies$ f ottenuta per minimizzazione limitata da R è computabile.
Dimostrazione:
Sia $\mathcal{M}$ algoritmo di decisione per R. Costruiamo un algoritmo di calcolo per f:
Su input $(\vec{x}, y) \in \mathbb{N}^{k+1}$:

- Ciclo FOR: per z da 0 a y:
  - Decido se $(\vec{x}, z) \in R$ (usando $\mathcal{M}$)
  - Se sì, restituisco z
- Se il ciclo finisce senza aver trovato niente, restituisco 0.

Proposizione: R rp $\implies$ f ottenuta per minimizzazione limitata da R è rp.
Dimostrazione:
Vogliamo trovare uno schema di Ricorsione Primitiva con f:

$$\begin{cases} f(\vec{x}, 0) = g(\vec{x}) \\ f(\vec{x}, y+1) = \varphi(\vec{x}, y, f(\vec{x}, y)) \end{cases}$$

- $f(\vec{x}, 0) = 0 = C_0^{(k)}(\vec{x})$ (che è rp)
- $f(\vec{x}, y+1) = \begin{cases} f(\vec{x}, y), & \text{SE } \exists\ z,\ 0 \leq z \leq y \text{ t.c. } (\vec{x}, z) \in R \\ y+1, & \text{SE NON SONO NEL CASO PRECEDENTE E } (\vec{x}, y+1) \in R \\ 0, & \text{ALTRIMENTI} \end{cases}$

Dunque definiamo φ:

$$\varphi(\vec{x}, y, t) = \begin{cases} t = \mathcal{E}_{k+2}^{(k+2)}(\vec{x}, y, t) & \text{SE } (\vec{x}, y, t) \in R_1 \\ y+1 = (S \circ \mathcal{E}_{k+1}^{(k+2)})(\vec{x}, y, t) & \text{SE } (\vec{x}, y, t) \in R_2 \\ 0 = C_0^{(k+2)}(\vec{x}, y, t) & \text{ALTRIMENTI} \end{cases}$$

Dove:

$$R_1 = \{(\vec{x}, y, t) \mid \exists\ z,\ 0 \leq z \leq y : (\vec{x}, z) \in R\}$$
$$R_2 = \{(\vec{x}, y, t) \mid (\vec{x}, y, t) \notin R_1 \text{ e } (\vec{x}, y+1) \in R\}$$

Per costruzione, $f(\vec{x}, y+1) = \varphi(\vec{x}, y, f(\vec{x}, y))$. Resta da mostrare che φ è rp. Ricordiamo che, per ipotesi, R è rp, dunque χ_R è rp.
Analizziamo R_1:

$$\chi_{R_1}(\vec{x}, y, t) = sg\left(\underbrace{\sum_{z=0}^{y} \chi_R(\vec{x}, z)}_{\text{operatore di somma limitata applicato a } \chi_R \text{ rp } \implies \text{ rp}}\right)$$

Essendo sg e la somma limitata entrambe operazioni che mantengono la ricorsività primitiva, χ_{R_1} è rp.
Sia $T = \{(\vec{x}, y, t) \in \mathbb{N}^{k+2} \mid (\vec{x}, y+1) \in R\}$.

$$\chi_T(\vec{x}, y, t) = \chi_R(\vec{x}, y+1) = \left(\chi_R \circ (\mathcal{E}_1^{(k+2)}, \ldots, \mathcal{E}_k^{(k+2)}, S \circ \mathcal{E}_{k+1}^{(k+2)})\right)(\vec{x}, y, t)$$

Da cui χ_T è rp.
Infine per R_2:

$$R_2 = R_1^c \cap T$$

Poiché R_1, e di conseguenza R_1^c, sono rp, e T è rp, anche R_2 è rp.

2.2.3 Funzione di Ackermann e funzioni μ-ricorsive

Proposizione: l'insieme delle funzioni rp è **enumerabile**. (si può scrivere un algoritmo che produce una dopo l'altra tutte le funzioni rp).
Dimostrazione: f funzione rp
cioè ha una drp: $f_1, f_2, \ldots, f_n = f$
Posso codificare f usando le funzioni iniziali tramite cui f è costruita usando composizione e RP.

$$\Sigma = \{C, 0, 1, \ldots, 9, S, \mathcal{E}, \text{ ',' }, \circ, (,), R, \langle, \rangle\}$$

(Nota: con C e le cifre codifichiamo una delle funzioni costanti; la virgola serve per separare i parametri; $\circ$ indica la composizione; $\langle\rangle$ si usano per rappresentare l'inizio e la fine della codifica di una funzione)

Regole di codifica:

- $C_0^{(k)} \rightsquigarrow Ck$ (es. $C_0^{(27)} \rightsquigarrow C27$)
- $S \rightsquigarrow S$
- $\mathcal{E}_j^{(k)} \rightsquigarrow \mathcal{E}k, j$ (es. $\mathcal{E}_7^{(12)} \rightsquigarrow \mathcal{E}12, 7$)
- $f \circ (g_1, \ldots, g_n) \rightsquigarrow \langle\langle f\rangle \circ (\langle g_1\rangle, \ldots, \langle g_n\rangle)\rangle$
- f ottenuta per RP da g e φ:
  $f = R(g, \varphi) \rightsquigarrow R(\langle g\rangle, \langle\varphi\rangle)$

Es:

$$S = R(\mathcal{E}_1^{(1)}, S \circ \mathcal{E}_3^{(3)}) \rightsquigarrow \langle R(\langle\mathcal{E}1, 1\rangle, \langle\langle S\rangle \circ (\langle\mathcal{E}3, 3\rangle)\rangle)\rangle$$

Proposizione: $\exists$ una funzione computabile unaria che non è rp.
Dimostrazione:
Per assurdo, supponiamo che ogni funzione computabile unaria sia rp.
Per la proposizione precedente (sulla codifica), l'insieme delle funzioni rp unarie è **enumerabile**.
Cioè $\exists$ un algoritmo di enumerazione che produce la sequenza:

$f_0, f_1, f_2, \ldots, f_x, \ldots$ (lista delle funzioni rp unarie prodotte dall'algoritmo di enumerazione)

Definisco la funzione successore diagonale: $g(x) = f_x(x) + 1$ dove $+ 1$ indica la funzione successore.
Osservazione:

- g è computabile perché f_x e S sono computabili.
- $\exists\ u$ t.c. $g = f_u$ (segue dall'ipotesi iniziale che ogni funzione computabile unaria è rp, e quindi è presente nella lista di enumerazione).

Quanto fa $g(u)$? Dalla definizione di g: $g(u) = f_u(u) + 1$
Ma sapendo che $g = f_u$, si ha: $g(u) = f_u(u)$

$$\Longrightarrow \textbf{ ! ASSURDO ! }$$

FUNZIONE DI ACKERMANN
$A : \mathbb{N}^2 \to \mathbb{N}$

$$\begin{cases} A(0, y) = y + 1 \\ A(x + 1, 0) = A(x, 1) \\ A(x + 1, y + 1) = A(x, A(x + 1, y)) \end{cases}$$

(Dimostra che non tutte le funzioni calcolabili sono rp, e che esistono funzioni che crescono più velocemente di qualsiasi funzione rp).

Esempi:

$$\begin{aligned} A(1, 1) &= A(0, A(1, 0)) \\ &= 1 + A(1, 0) \\ &= 1 + A(0, 1) \\ &= 1 + 2 = 3 \end{aligned}$$

$$
\begin{aligned}
A(2,4) &= A(1,A(2,3)) = 11 \\
A(2,3) &= A(1,A(2,2)) \\
A(2,2) &= A(1,A(2,1)) = A(1,5) = A(0,A(1,4)) \\
A(2,1) &= A(1,A(2,0)) = A(1,A(1,1)) = A(1,3) \\
&= A(0,A(1,2)) \\
&= 1 + A(1,2) \\
&= 1 + A(0,A(1,1)) \\
&= 1 + 4 = 5
\end{aligned}
$$

È una funzione che cresce molto per anche aumenti minimi dei parametri.

- $A(0,y) = y+1$
- $A(1,y) = y+2$
  Dimostrazione per induzione su y:
  - Base: $A(1,0) = A(0,1) = 2 = 0+2$
  - Passo: Supponiamo che $A(1,y) = y+2$.
  $A(1,y+1) = A(0,A(1,y)) = 1 + A(1,y) = 1+y+2 = y+3 = (y+1)+2$
- $A(2,y) = 2y+3$
- $A(3,y) = 2^{y+3} - 3$
- $A(4,y) = \underbrace{2^{2^{\cdot^{\cdot^{\cdot^{2^{16}}}}}}}_{y \text{ volte}} - 3$

PROPRIETÀ DI A

1. $A(x,y) \geq y+1$
2. $A(x,y) < A(x,y+1)$
   Dimostrazione per induzione su x:
   - $x = 0$: $A(0,y) = y+1 < y+2 = A(0,y+1)$
   - Supponiamo che $A(x,y) < A(x,y+1)$
     $A(x+1,y) \underset{\text{per Prop 1}}{\leq} A(x+1,y)+1 \leq A(x,A(x+1,y)) = A(x+1,y+1)$
3. $A(x,y+1) \leq A(x+1,y)$
4. $A(x,y) < A(x+1,y)$
5. $A(x_1,y) + A(x_2,y) < A(\max(x_1,x_2)+4,y)$ (qui la cosa importante è notare come si passi due parametri x_1 e x_2 a un unico parametro $\max(x_1,x_2)+4$)
6. $A(x,y) + y \leq A(x+4,y)$

Proposizione: $\forall\ g : \mathbb{N}^k \to \mathbb{N}$ rp, $\exists\ c \in \mathbb{N}$:

$$\forall\ \vec{x} \in \mathbb{N}^k, \quad g(\vec{x}) < A\left(c, \sum_{i=1}^{k} x_i\right)$$

Dimostrazione: Per induzione strutturale sulla costruzione di g

- Funzioni iniziali:

- $C_0^{(k)}(\vec{x}) = 0 < 1 + \sum_{i=1}^{k} x_i = A\left(\overset{c=0}{0}, \sum_{i=1}^{k} x_i\right)$
- $S(x) = x + 1 < x + 2 = A(0, x+1) \overset{(3)}{\leq} A\left(\underset{(c=1)}{1}, x\right)$
- $\mathcal{E}_j^{(k)}(\vec{x}) = x_j < 1 + \sum_{i=1}^{k} x_i = A\left(\underset{c=0}{0}, \sum_{i=1}^{k} x_i\right)$

$\implies$ La proposizione vale per le funzioni iniziali.

- **Composizione:** $g(\vec{x}) = h \circ (f_1(\vec{x}), \dots, f_n(\vec{x}))$
  Supponiamo che:
  - $h(x_1, \dots, x_n) < A\left(D, \sum_{i=1}^{n} x_i\right) \quad (\exists\ D \in \mathbb{N})$
  - $\forall\ j = 1 \dots n, \quad f_j(\vec{x}) < A\left(C_j, \sum_{i=1}^{k} x_i\right) \quad (\exists\ C_j \in \mathbb{N})$

  Calcoliamo:

$$
\begin{aligned}
g(\vec{x}) = h(f_1(\vec{x}), \dots, f_n(\vec{x})) &< A\left(D, \sum_{i=1}^{n} f_i(\vec{x})\right) \\
&< A\left(D, \sum_{i=1}^{n} A\left(C_j, \sum_{i=1}^{k} x_i\right)\right) \\
&\underset{\text{iterando la (5)}}{\leq} A\left(D, A\left(\widetilde{C}, \sum_{i=1}^{k} x_i\right)\right) \quad \text{dove } \widetilde{C} = \max_i(C_i) + 4 \\
&\leq A\left(E, A\left(E+1, \sum_{i=1}^{k} x_i\right)\right) \\
&= A\left(E+1, \sum_{i=1}^{k} x_i + 1\right) \\
&\leq A\left(E+2, \sum_{i=1}^{k} x_i\right)
\end{aligned}
$$

Proposizione: A non è rp.
Dimostrazione:
Supponiamo per assurdo che A sia rp.
Allora la funzione $B(x) = A(x, x)$ è rp
(poiché $B(x) = (A \circ (\mathcal{E}_1^{(1)}, \mathcal{E}_1^{(1)}))(x)$)
Dunque per B deve valere la proposizione precedente:

$$\implies \forall\ x \in \mathbb{N}, \exists\ c \in \mathbb{N} \text{ t.c. } B(x) < A(c, x)$$

Calcolo $B(c)$:

$$A(c, c) = B(c) < A(c, c) \quad \textbf{! ASSURDO !}$$

2.2.4 Funzioni μ-ricorsive

Aggiungiamo un costrutto per estendere la classe delle funzioni rp, in modo da riuscire a catturare anche A e altre funzioni che crescono più velocemente di qualsiasi funzione rp.

Definizione: $g : \mathbb{N}^{k+1} \to \mathbb{N}$ si dice **REGOLARE** quando:

$$\forall\ \vec{x} \in \mathbb{N}^k,\ \exists\ y \in \mathbb{N} : g(\vec{x}, y) = 0$$

Definizione: $f : \mathbb{N}^k \to \mathbb{N}$ si dice **OTTENUTA PER MINIMIZZAZIONE** da $g : \mathbb{N}^{k+1} \to \mathbb{N}$ regolare quando:

$$f(\vec{x}) = \min\{y \in \mathbb{N} \mid g(\vec{x}, y) = 0\}$$

Definizione: UNA **DERIVAZIONE μ-RICORSIVA** È UNA SEQUENZA di funzioni $f_1, f_2, \ldots, f_n$ tale che $\forall\ i = 1, \ldots, n$:

- f_i è FUNZIONE INIZIALE, oppure
- f_i è OTTENUTA PER COMPOSIZIONE DA FUNZIONI PRECEDENTI, oppure
- f_i è OTTENUTA PER RP DA 2 FUNZIONI PRECEDENTI, oppure
- f_i è OTTENUTA DA f_j REGOLARE PER MINIMIZZAZIONE (con $j < i$, ovvero f_j è precedente a f_i).

Definizione: UNA FUNZIONE SI DICE **μ-RICORSIVA** QUANDO COMPARE IN CODA A UNA DERIVAZIONE μ-RICORSIVA.

Proposizione: g computabile $\implies$ f computabile *(riferito a queste g ed f)*
Dimostrazione:
Sia $\mathcal{G}$ algoritmo di calcolo per g.
Algoritmo di calcolo per f:

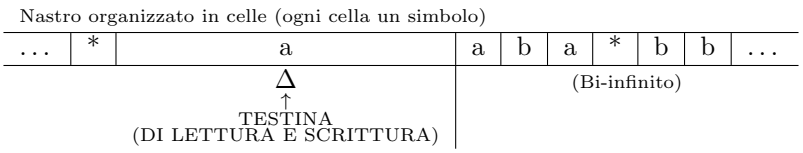
- Data $\vec{x} \in \mathbb{N}^k$
- mentre $y \geq 0$:
- $\left.\begin{array}{l}\text{Calcolo } g(\vec{x}, y) \text{ (usando } \mathcal{G}) \\ \text{Se ottengo 0, restituisco } y\end{array}\right\}$ *(Ciclo While)*

Ci sono indizi che ci fanno pensare che è una classe robusta (più della rp):

- La funzione di Ackermann è μ-ricorsiva.
- Non conosciamo (ad oggi) alcun controesempio.
- Ci sono anche altre proposte di definizioni (ed è quindi molto improbabile che tutte queste definizioni, che sono diverse tra loro, siano tutte sbagliate).

Tesi di Church (per le funzioni μ-ricorsive)
La classe delle funzioni computabili coincide con la classe delle funzioni μ-ricorsive.
(Tesi e non Teorema perché non c'è dimostrazione matematica formale dell'idea di "calcolabile")

2.3 Macchine di Turing



Ogni cella è predisposta per accogliere un simbolo.

$\Sigma \cup \{*\}$ Alfabeto.
Ad ogni istante la macchina, in base a quello che legge e allo stato in cui si trova, può fare una di queste tre operazioni:

- Sovrascrivere (aggiornare) il simbolo
- Spostarsi (a destra o a sinistra)
- Aggiornare lo stato

Q insieme degli stati $\{q_0, q_1, q_2, \dots\}$

Transizione (singolo passo di calcolo/computazione):

$$(\underbrace{q}_{\text{stato}}, \underbrace{X}_{\text{simbolo}}, \underbrace{\alpha}_{\text{azione}}, \underbrace{\tilde{q}}_{\text{stato di arrivo}}) \in Q \times \Sigma \times (\Sigma \cup \{D, S\}) \times Q$$

Dove $\alpha \in \Sigma \cup \{D, S\}$ indica:

- D, S: passo a destra, passo a sinistra
- Se $\alpha \in \Sigma$: scrivo sul nastro senza spostamento

Configurazione: È la coppia (q, X), dove:

- q è lo stato in cui si trova la macchina
- X è la stringa che rappresenta il contenuto del nastro

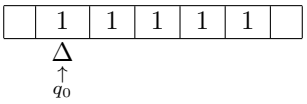
Quartupla funzionale nei primi 2 argomenti: Fissati i primi 2 argomenti (lo stato corrente e il simbolo letto), esiste al più una transizione. Questo garantisce che la macchina sia *deterministica*.

Definizione: Una **Macchina di Turing** (MdT) è una lista di transizioni (cioè una lista di quadruple funzionali nei primi 2 argomenti).

Esempi
1)

$q_0 \ * \ D \ q_1$

$q_0 \ 1 \ D \ q_1$



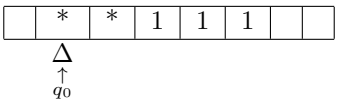
Questa macchina fa solo un passo a destra e si ferma.

2)

$q_0 \ * \ D \ q_0$

$q_0 \ 1 \ D \ q_1$

$q_1 \ 1 \ D \ q_1$



Legge gli asterischi spostandosi a destra; quando trova un 1*, passa allo stato* q_1 *e continua a scorrere gli* 1 *verso destra.*

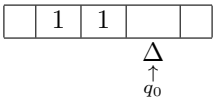
3)

$q_0 \ * \ 1 \ q_0$

$q_0 \ 1 \ D \ q_1$

$q_1 \ * \ 1 \ q_1$

$q_1 \ 1 \ 1 \ q_0$



Mette tutti 1 *e non termina mai.*
(Nota: si può scrivere anche come $q_0 \ * \ 1 \ q_0$ *seguito da* $q_0 \ 1 \ D \ q_0$*, poiché per fare la stessa cosa ci possono essere più Macchine di Turing.*

4) Funzione Successore

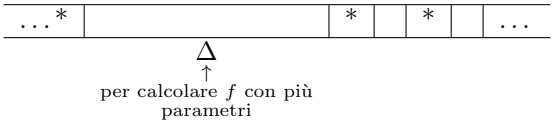
$q_0 \ * \ D \ q_1$

$q_1 \ 1 \ D \ q_1$

$q_1 \ * \ 1 \ q_2$

$q_2 \ 1 \ S \ q_2$

Definizione: Una MdT M **calcola una funzione** f quando, fatta partire sulla cella vuota all'immediata sinistra dell'input, termina sulla cella vuota all'immediata sinistra dell'output.



Cioè si inserisce degli * per delimitare i parametri.

5) Funzione Somma

$q_0 \ * \ D \ q_1$

$q_1 \ 1 \ D \ q_1$

$q_1 \ * \ 1 \ q_2$

$q_2 \ 1 \ D \ q_2$

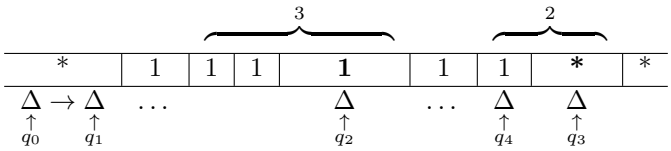
$q_2 \ * \ S \ q_3$

$q_3 \ * \ S \ q_4$

$q_4 \ 1 \ * \ q_4$

$q_4 \ * \ S \ q_5$

$q_5 \ 1 \ S \ q_5$



(Nota: La macchina scorre a destra trasformando l'asterisco centrale in un 1*, unendo così i due blocchi. Arrivata in fondo, cancella gli ultimi due* 1 *sostituendoli con degli* $*$ *per compensare l'eccesso della codifica unaria e mantenere il risultato corretto della somma. Infine, torna all'indietro fino all'asterisco iniziale per fermarsi).*

Funzione τ-ricorsiva: Una funzione $f : \mathbb{N}^k \to \mathbb{N}$ si dice τ-ricorsiva quando $\exists$ M MdT che calcola f.

Tesi di Church (per funzioni τ-ricorsive): La classe delle funzioni computabili coincide con la classe delle funzioni τ-ricorsive.

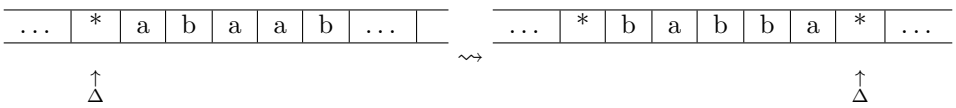
Esercizio
Siano $f : \mathbb{N} \to \mathbb{N}$ rp, $R \subseteq \mathbb{N}^k$ rp (quindi χ_R è rp).
Definisco $S \subseteq \mathbb{N}^k$ ponendo $\vec{x} \in S$ usando $(f(x_1), \dots, f(x_k)) \in R$.
Dimostrare che S è rp.

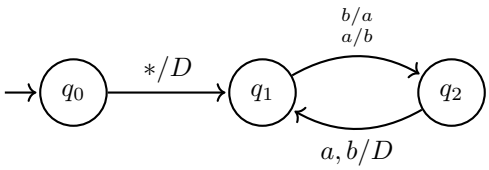
Dimostrazione:

$$\chi_S(\vec{x}) = \chi_R \circ (f \circ \mathcal{E}_1^{(k)}, \dots, f \circ \mathcal{E}_k^{(k)})(\vec{x})$$

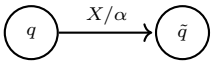
Essendo S definita tramite la composizione della funzione caratteristica di R (che è rp per ipotesi) con una serie di composizioni di f (rp per ipotesi) e le proiezioni (rp in quanto funzioni iniziali), la funzione χ_S è ottenuta per composizione di funzioni rp, ed è quindi essa stessa rp. Di conseguenza, l'insieme S è rp.

6) MDT che, data una stringa su $\{a, b\}$, scambia le a con le b





Rappresentazione grafica di una MDT:



7) MDT che scrive la copia di una stringa unaria

...	*	1	1	1	1	*	...	

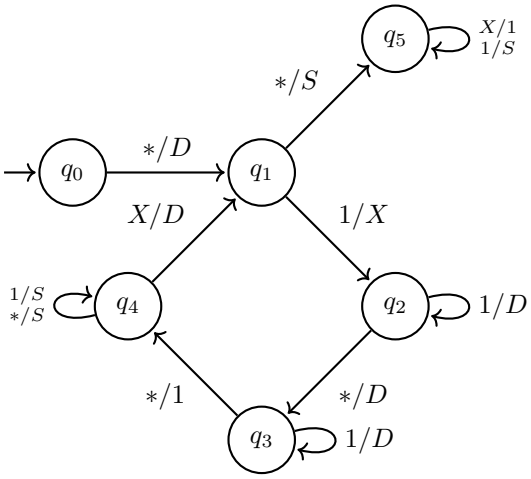
$\uparrow$ q_0

$\rightsquigarrow$

...	*	1	1	1	1	*	1	1	1	1	*	...

$\uparrow$ q_5

Nota: si utilizza un simbolo aggiuntivo (di lavoro) X per indicare la copia di un 1.



(Nota: la macchina legge un 1*, lo sostituisce con una X e scorre a destra. Salta l'asterisco separatore e gli eventuali* 1 *già copiati, fino a trovare la prima cella vuota ($*$) dove scrive un* 1*. Poi torna indietro verso sinistra fino a ritrovare la X per ripetere il processo. Quando non ci sono più* 1 *da copiare, la macchina entra nella fase di ripristino: scorre verso sinistra sostituendo ogni X con un* 1 *per ricreare l'input originale, finché non incontra l'asterisco iniziale, dove si ferma definitivamente).*

2.3.1 Macchine di Turing come accettatori di linguaggi

Sia Q l'insieme degli stati di una MDT e $F \subseteq Q$ l'insieme degli stati finali.

Definizione: Una macchina M accetta la stringa w per stati finali quando l'esecuzione di M su input w termina in uno stato finale.

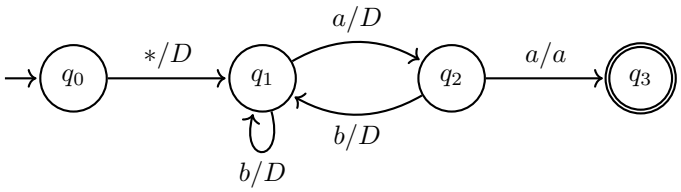
Definizione: Un linguaggio $L \subseteq \Sigma^*$ si dice accettato per stati finali da una MDT M quando: $w \in L \iff M$ accetta w per stati finali. In questo caso si scrive $L = L(M)$.

Definizione: Se L è tale che $\exists M$ MDT per cui $L = L(M)$, l'insieme L si dice **ricorsivamente enumerabile**.

Osservazione: Se M è una MDT che termina su ogni input, allora $L(M)$ si dice **ricorsivo**.

Via Tesi di Church:
ricorsivamente enumerabile $\iff$ semidecidibile
ricorsivo $\iff$ decidibile

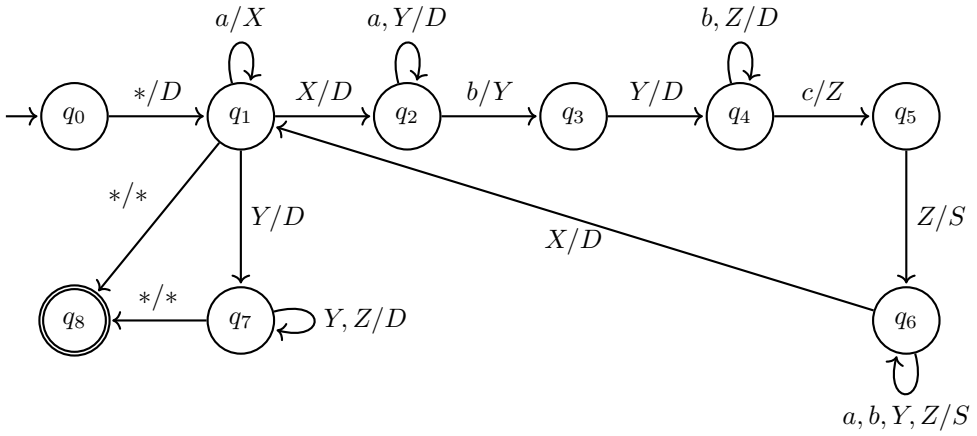
Esempio 1) MDT che accetta il linguaggio $(a|b)^*aa(a|b)^*$



$Q = \{q_0, q_1, q_2, q_3\}, \quad F = \{q_3\}$

(Nota: la macchina, dopo aver trovato due a consecutive, entra nello stato finale q_3 e si ferma, accettando così la stringa a prescindere da quello che c'è dopo le due a).

Esempio 2) MDT che accetta il linguaggio $\{a^i b^i c^i \mid i \geq 0\} \subseteq \{a, b, c\}^*$



Nota esplicativa sul funzionamento:
La macchina non potendo "contare" la lunghezza i, procede accoppiando i simboli a tre a tre tramite passate successive, utilizzando i simboli ausiliari X, Y, Z come marcatori.

- ***Ciclo di marcatura*** *($q_1 \to q_6$)**:*** *Partendo da sinistra, la macchina legge la prima a disponibile e la marca scrivendo X. Scorre a destra saltando le a e le Y (q_2), trova la prima b e la marca con Y (q_3). Scorre ancora a destra saltando le b e le Z (q_4), trova la prima c e la marca con Z (q_5). A questo punto inverte la marcia (q_6) e torna tutta a sinistra fino a sbattere contro l'ultima X scritta, per poi fare un passo a destra e ricominciare un nuovo ciclo.*
- ***Fase di verifica*** *(q_7)**:*** *Quando in q_1 la macchina legge una Y invece di una a, significa che le a sono finite. Entra quindi in fase di verifica (q_7): scorre tutto il nastro a destra assicurandosi che siano rimaste solo Y e Z (nessuna b o c in eccesso).*
- ***Accettazione*** *(q_8)**:*** *Se arriva in fondo leggendo solo marcatori e incontra l'asterisco finale ($*$), la stringa è perfettamente bilanciata e viene accettata per arresto nello stato finale q_8. La transizione $q_1 \xrightarrow{*/*} q_8$ serve invece per accettare la stringa vuota (caso $i = 0$).*

Definizione: Una MDT M accetta w **per arresto** quando M, eseguita su w, termina. (in qualsiasi stato).

Definizione: L'insieme delle stringhe accettate da M per arresto è il linguaggio accettato da M per arresto.

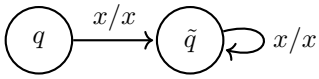
Proposizione: Sia L un linguaggio. $\exists M$ MDT che accetti L per stati finali $\iff$ $\exists N$ MDT che accetta L per arresto.
Dimostrazione:

$\Longleftarrow$) Sia N una MDT che accetta L per arresto. La MDT M ottenuta da N designando ogni stato come stato finale è una MDT che accetta L per stati finali.

$\Longrightarrow$) Sia M una MDT che accetta L per stati finali. Su input w:

- M termina in uno stato finale (selezionando Q = stati di M, F = stati finali di M)
- M termina in uno stato non finale
- M non termina

Costruiamo la MDT N in questo modo: N ha le stesse transizioni di M, più le seguenti: $\forall q \in Q \setminus F$, $\forall x \in \Sigma$ tali che non ci sono transizioni di M con configurazione (q, x), aggiungiamo le transizioni $q \xrightarrow{x/x} \tilde{q}$ e $\tilde{q} \xrightarrow{x/x} \tilde{q}$ (dove $\tilde{q}$ è un nuovo stato in cui la macchina "cicla" all'infinito senza mai fermarsi).



Esercizi

1) L è accettato per un unico stato finale quando $\exists M$ MDT con un solo stato finale che accetta L. Dimostrare che L è accettato per stati finali $\iff$ è accettato per un unico stato finale.

Dimostrazione:

$\Longleftarrow$) Ovvio.

$\Longrightarrow$) Sia M una MDT che accetta L per stati finali. Costruiamo N aggiungendo un nuovo stato $\tilde{q}$, che M raggiungerà ogni volta che termina in uno stato finale (inoltre $\tilde{q}$ sarà l'unico stato finale di N).

2) M accetta w **per ingresso** quando, durante l'esecuzione di M su w, la MDT entra in uno stato finale. Un linguaggio L è accettato da M per ingresso quando $\exists M$ MDT che accetta tutte e sole le stringhe di L per ingresso. Dimostrare che L è accettato per stati finali $\iff$ L è accettato per ingresso.

Dimostrazione:

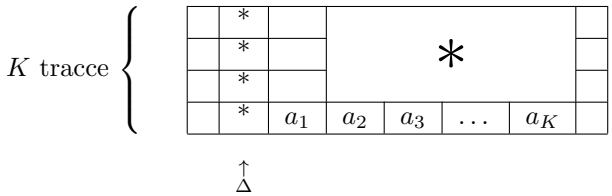
$\Longrightarrow$) Sia M una MDT che accetta L per stati finali. M' che accetta per ingresso si costruisce a partire da M aggiungendo un nuovo stato $\tilde{q}$ (che sarà l'ultimo stato finale) e transizioni che portano in $\tilde{q}$ da ogni stato finale in corrispondenza a caratteri per cui non ci sono transizioni uscenti in M.

$\Longleftarrow$) Sia M una MDT che accetta L per ingresso. M' MDT che accetta L per stati finali si ottiene da M eliminando tutte le transizioni uscenti dagli stati finali.

2.3.2 Varianti di Macchine di Turing

MDT multitraccia: Ha Σ alfabeto, Q insieme degli stati e una lista di transizioni della forma:

$$(q, (x_1, \ldots, x_k), \alpha, \tilde{q}) \in Q \times \Sigma^k \times (\Sigma^k \cup \{D, S\}) \times Q$$



Cioè la testina legge un vettore di k simboli.

Proposizione: Fissato $K \in \mathbb{N}$, L è accettato da una MDT a una traccia $\iff$ L è accettato da una MDT a K tracce.

Dimostrazione:

$\Longrightarrow$) Ovvio (posso simulare una MDT con una traccia, usando una MDT con il contenuto di K tracce vuote, tranne la prima).

$\Longleftarrow$) Sia M una MDT a K tracce che accetta L. Posso ottenere una MDT M' che accetta L a una traccia semplicemente sostituendo l'alfabeto Σ di M con l'alfabeto esteso Σ^K (le cui lettere sono di fatto K-uple), ed eseguendo le stesse transizioni.

MDT limitata a sinistra:

*	a_1	...	a_n	...

$\uparrow$
Δ

Possiamo usare una MDT limitata a sinistra per accettare stringhe nello stesso modo di quelle classiche, con l'accortezza che un'operazione di spostamento a sinistra a partire dalla prima cella causa il rifiuto della stringa.

Proposizione: L accettato da una MDT classica $\iff$ L accettato da una MDT limitata a sinistra.
Idea di Dimostrazione:

$\Longleftarrow$) **Da Limitata a Classica:** Sia M una MDT limitata a sinistra che accetta L. Vogliamo simularla con una MDT classica (a nastro bi-infinito). La macchina classica farà le stesse identiche cose, ma posizionerà un simbolo speciale (ad esempio #) nella cella di partenza (indice 0). Se durante la computazione la macchina legge questo simbolo e riceve l'istruzione di spostarsi a sinistra, saprà che la mossa è "illegale" e rifiuterà la stringa, simulando esattamente il blocco della macchina limitata.

$\Longrightarrow$) **Da Classica a Limitata:** Sia M una MDT classica che accetta L. Immaginiamo di numerare le celle del suo nastro infinito partendo dalla cella di partenza come indice 0 (avremo celle positive a destra $1, 2, 3\ldots$ e negative a sinistra $-1, -2, -3\ldots$). Creiamo una MDT limitata a sinistra dotata di **2 tracce**, che funziona come se avessimo "piegato a metà" il nastro originale proprio sullo zero:

	-0	-1	-2	-3		
Traccia 2 (indici negativi) $\rightarrow$						...
Traccia 1 (indici ≥ 0**)** $\rightarrow$	*	a_1	a_2	a_3		...
	0	1	2	3		

La nuova macchina replica il comportamento di M per le celle ≥ 0 operando sulla prima traccia (quella inferiore). Quando la macchina originale cercherebbe di passare a sinistra dello 0 (verso gli indici negativi), la nostra nuova macchina passa a operare sulla seconda traccia (quella superiore) e **inverte gli spostamenti**: poiché il nastro è ripiegato, un passo a "sinistra" della macchina originale diventa un passo a "destra" sulla nostra traccia ripiegata, e viceversa.

NB: MDT standard $\iff$ MDT limitata a sinistra.

MDT multinastro: Opera su K nastri, ciascuno dotato della propria testina di lettura/scrittura. Le K testine possono operare e muoversi in modo totalmente indipendente l'una dall'altra. Una transizione di una MDT multinastro è della forma:

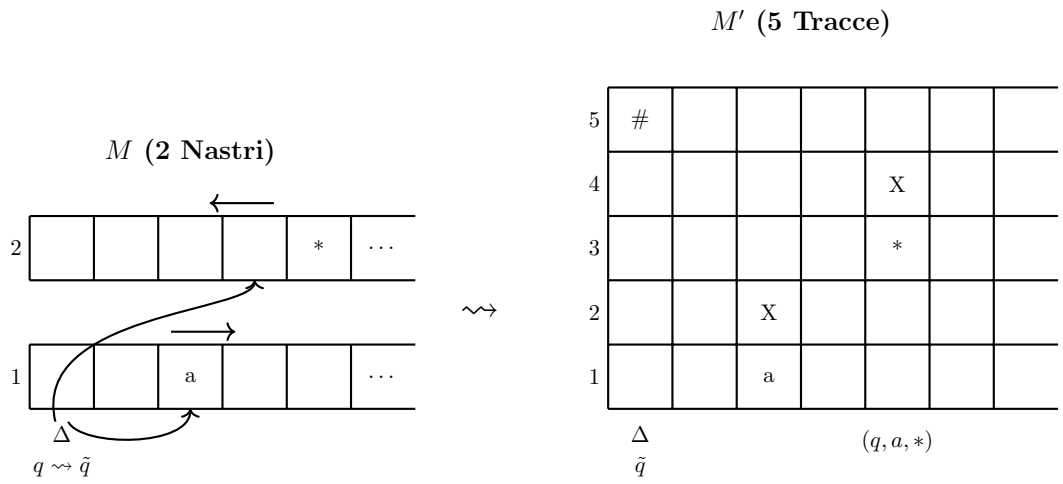
$$(q, (x_1, \ldots, x_k), (\alpha_1, \ldots, \alpha_k), \tilde{q}) \in Q \times \Sigma^K \times (\Sigma \cup \{D, S\})^K \times Q$$

Proposizione: L accettato da una MDT standard $\iff$ L accettato da una MDT multinastro.
Dimostrazione:

$\Longrightarrow$) Ovvio (basta ignorare i nastri in eccesso).

$\Longleftarrow$) Sia M una MDT a K nastri che accetta L. Facciamo vedere che $\exists$ una MDT M' a $2K+1$ tracce che accetta L. Più precisamente facciamo vedere che ogni singola transizione di una computazione di M può essere simulata da un gruppetto di transizioni di M' (usando una traccia per il contenuto di ogni nastro e una traccia ausiliaria per marcare la posizione di ogni singola testina).

Transizioni di M' (Simulazione di una MDT a K nastri con una a $2K+1$ tracce):
Esempio con $K = 2$: La MDT M ha 2 nastri indipendenti, quindi la MDT M' che la simula avrà $2(2) + 1 = 5$ tracce.



Come funziona la simulazione (su M'):

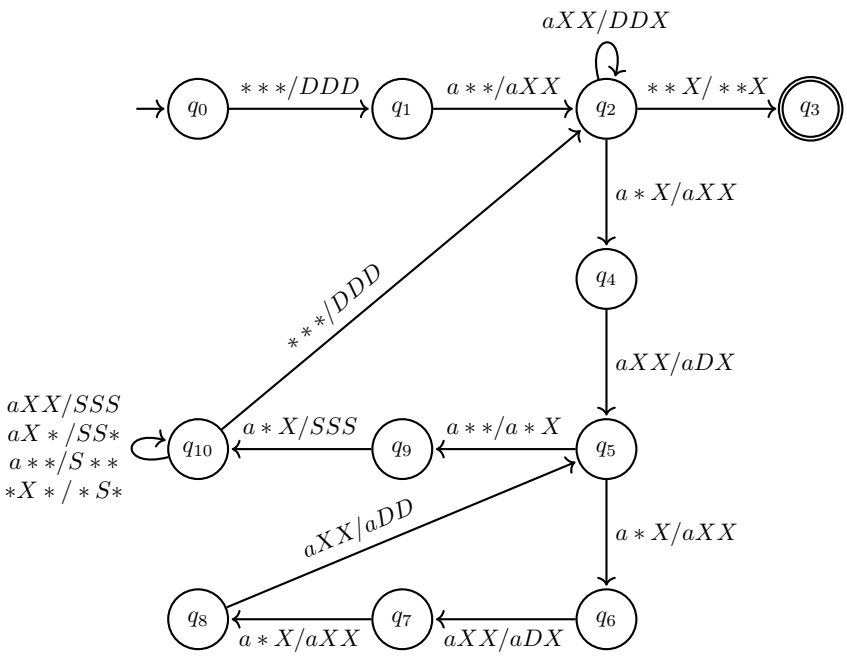
- La prima cosa che fa è inserire un simbolo speciale (#) nell'ultima traccia (la traccia 5) per marcare l'inizio.
- La prima traccia viene usata per il contenuto del primo nastro.
- La terza traccia viene usata per il contenuto del secondo nastro.
- La seconda e la quarta traccia memorizzano la posizione delle rispettive testine (segnando una X nella colonna in cui si troverebbe la testina).
- Per eseguire una transizione, parto sempre con la testina all'inizio (sul #).
- Scorro in avanti, guardo la X di traccia 2 e memorizzo il simbolo associato in traccia 1 (in questo caso a).
- Torno indietro fino al # e faccio la stessa cosa per il secondo nastro (cercando la X in traccia 4 per memorizzare il simbolo in traccia 3).
- Torno di nuovo all'inizio, e poi faccio le scritture e il movimento della X allo stesso modo (sempre andando avanti e indietro a partire dal #).

Nota: Per memorizzare i simboli letti durante questi passaggi, l'idea è utilizzare la memoria interna degli stati. Definiamo gli stati di M' come tuple, ad esempio $(q, a, *)$, che "ricordano" sia lo stato originale q in cui ci si trovava, sia i simboli appena letti.

Esempio: MDT a 3 nastri che decide il linguaggio (su $\{a\}$) $\{a^K \mid K$ è un quadrato perfetto$\}$

Nastro 1 (K):		X	X					
Nastro 2 (K^2):		X	X	X	X			
Nastro 3 (Input):	*	a	a	a	a	a	a	...
	↑ Δ							

Logica matematica per l'aggiornamento: $K^2 \rightsquigarrow (K+1)^2 = K^2 + 2K + 1$



2.4 MDT Non Deterministica

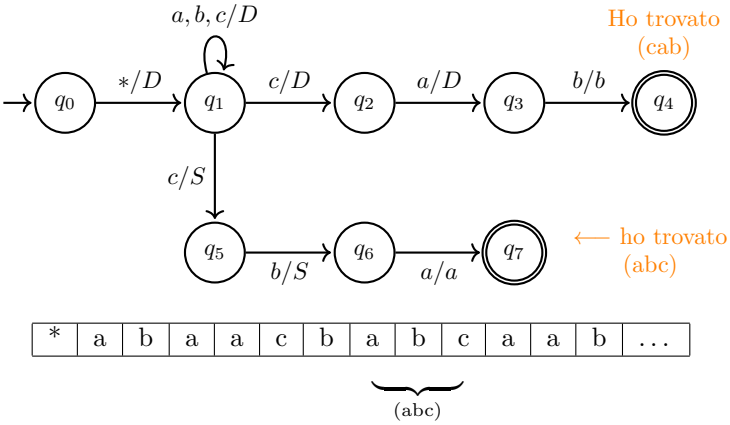
Definizione: Una MDT si dice **non deterministica** quando le transizioni non sono necessariamente funzionali nei primi 2 argomenti. (Ovvero, a parità di stato e simbolo letto, la macchina può avere più scelte su cosa fare).

Osservazione: Le MDT deterministiche (= standard) sono MDT non deterministiche (ovvero, con una sola scelta per ogni configurazione).

Definizione: Data una stringa w e una MDT non deterministica M, diciamo che M accetta w quando **esiste** (almeno) una computazione di M che accetta w.

Esempio:

$$L = \{w \in \{a,b,c\}^* \mid \exists \text{ un'occorrenza di 'c' immediatamente preceduta da } ab \text{ (abc)}$$
$$\text{oppure immediatamente seguita da } ab \text{ (cab)}\}$$



Grado di non determinismo di M: Data M MDT non deterministica, il suo grado di non determinismo δ è il massimo numero di scelte possibili per una singola configurazione:

$$\delta = \max_{\substack{\tilde{q}\in Q \\ \tilde{x}\in\Sigma}} |\{\tilde{q}\tilde{x}\alpha q \mid \alpha \in \Sigma \cup \{D, S\}, q \in Q\}|$$

Dato $q \in Q, x \in \Sigma$ codifichiamo le transizioni di M (MDT non deterministica) aventi qx come primi 2 elementi utilizzando gli interi da 1 a δ, possibilmente codificando la stessa transizione con più etichette.

- $q_1\ c\ D\ q_1\ :\ 1 \qquad q_1\ a\ D\ q_1\ :\ 1, 2, 3$
- $q_1\ c\ D\ q_2\ :\ 2$
- $q_1\ c\ S\ q_5\ :\ 3$

Esempi di possibili computazioni sulla stringa $w = acab$:

1. 1111, termina in uno stato non finale (w *non accettata*)
2. 11211, termina in uno stato finale (w **accettata**)
3. 22322, termina subito (w *non accettata*)

Proposizione: L è accettato da una MDT standard $\iff$ L è accettato da una MDT non deterministica.
Dimostrazione:

$\Longrightarrow$) Sia M una MDT standard che accetta L. Poiché ogni MDT deterministica è (banalmente) anche una MDT non deterministica, L è accettato da una MDT non deterministica.

$\Longleftarrow$) Sia M una MDT non deterministica che accetta L per arresto, con grado di non determinismo di $M = \delta$. Descriviamo una MDT *deterministica* che accetta L per arresto a 3 nastri (per simulare tutte le computazioni parallele di M in modo sistematico):

3	m_1	*	m_2	*	...	Generazione delle computazioni di M ($1 \leq m_i \leq \delta$)
2	*	copio l'input ed eseguo ($m_1, \ldots, m_K$)				Simulazione delle computazioni di M
1						Input

2.5 Funzioni Parziali Computabili

Funzione parziale: Dati A, B insiemi, una funzione parziale da A a B è una funzione da D a B, per qualche dominio $D \subseteq A$. *(Cioè potrebbe essere definita solo su una parte di A, ad es. $f(x) = \frac{10}{x}$, che non è definita per $x = 0$).*

Funzione parziale computabile: Data una funzione parziale f da A a B, si dice computabile quando $\exists M$ algoritmo tale che $\forall x \in A$:

- Se $x \in$ Dominio(f), M eseguita su x restituisce in output $f(x) \longrightarrow x \in$ Dominio(f): f converge su x, si scrive $f(x)\downarrow$
- Se $x \notin$ Dominio(f), M su x non termina $\longrightarrow x \notin$ Dominio(f): f diverge su x, si scrive $f(x)\uparrow$

Lavoriamo su $\mathbb{N}$ in codifica unaria.
Definizione: f funzione parziale da $\mathbb{N}^k \to \mathbb{N}$ si dice **parziale τ-ricorsiva** quando $\exists M$ MDT che, $\forall \vec{x} \in \mathbb{N}^k$:

- Se $f(\vec{x})\downarrow$ allora l'esecuzione di M su $\vec{x}$ termina con $f(\vec{x}) + 1$ "uni" sul nastro.
- Se $f(\vec{x})\uparrow$ allora l'esecuzione di M su $\vec{x}$ non termina su $\vec{x}$.

Tesi di Church per funzioni parziali τ-ricorsive: La classe delle funzioni parziali computabili coincide con la classe delle funzioni parziali τ-ricorsive.

Proposizione: $\forall M$ MDT deterministica, $\forall K \in \mathbb{N}$ $\exists!$ funzione parziale computabile $f : \mathbb{N}^k \to \mathbb{N}$ tale che M calcola f.

Dimostrazione: Definisco f funzione parziale da $\mathbb{N}^k$ in $\mathbb{N}$ come segue:
$\forall \vec{x} \in \mathbb{N}^k$, eseguo M su $\vec{x}$:

- Se M termina, $f(\vec{x})$ è data in unario dal numero di "uni" che si trovano sul nastro al termine dell'esecuzione.
- Se M non termina, $f(\vec{x}) \uparrow$. Questo ci dice che ad ogni M corrisponde (in modo univoco) una funzione f ($M \longleftrightarrow f$).

2.6 Enumerazione e Problema dell'Arresto

2.6.1 Codifica ed Enumerazione delle MDT

Proposizione: L'insieme delle Macchine di Turing (MDT) è enumerabile.

Dimostrazione: Possiamo codificare ogni Macchina di Turing come una stringa binaria.
Dato l'insieme degli stati $Q = \{q_0, q_1, q_2, \dots\}$ finito, associamo:

$$q_i \rightsquigarrow \underbrace{11\dots1}_{i+1} \quad \text{codifica}$$

Dato l'alfabeto $\Sigma = \{a_0, a_1, a_2, \dots, a_n\}$, associamo:

$$a_i \rightsquigarrow \underbrace{11\dots1}_{i+1}$$

Per i movimenti della testina:

$$D \rightsquigarrow \underbrace{11\dots1}_{n+2}$$

$$S \rightsquigarrow \underbrace{11\dots1}_{n+3}$$

La **transizione** generica $q_h\ a_i\ \alpha\ q_k$ (dove α è a_j oppure D oppure S) si codifica concatenando le sequenze di '1' separate da uno '0':

$$q_h\ a_i\ \alpha\ q_k \rightsquigarrow \underbrace{11\dots1}_{h+1}0\underbrace{11\dots1}_{i+1}0\underbrace{11\dots1}_{?}0\underbrace{11\dots1}_{k+1}$$

Lista delle transizioni: Per rappresentare l'intera macchina, concateniamo le singole transizioni (Tr.) separandole con "00" e terminando la stringa con "000":

"Transizione 1" 00 "Transizione 2" 00 ... 00 "Transizione z" 000

Nota: Il "000" finale segnala che finisce la codifica. Questa struttura binaria valida per qualsiasi Macchina di Turing è utile se si vuole dare in pasto ad un'altra Macchina di Turing.

Esempio di codifica: Date le seguenti transizioni:

$$q_0\ *\ D\ q_0$$
$$q_0\ 1\ D\ q_1$$
$$q_1\ 1\ S\ q_1$$

Traducendo i simboli secondo le regole (assumendo $1 \rightsquigarrow 1, * \rightsquigarrow 11$ e ricordando che $D \rightsquigarrow 111, S \rightsquigarrow 1111$), le singole transizioni diventano:

- $q_0\ *\ D\ q_0 \implies 1\ 0\ 11\ 0\ 111\ 0\ 1$

- $q_0\ 1\ D\ q_1 \implies 1\ 0\ 1\ 0\ 111\ 0\ 11$
- $q_1\ 1\ S\ q_1 \implies 11\ 0\ 1\ 0\ 1111\ 0\ 11$

Come buona prassi per delimitare chiaramente il codice (specialmente se dato in input a una Macchina Universale), apriamo e chiudiamo l'intera stringa con "000":

$$000\ 1011011101\ 00\ 1010111011\ 00\ 110101111011\ 000$$

Algoritmo di enumerazione per le MDT: Genera in ordine lessicografico e per lunghezza crescente tutte le stringhe su $\{0, 1\}$. Per ognuna di esse faccio un controllo sintattico e, se rappresenta una MDT, la scrivo. *(Esempio: per la sintassi, controlla che cominci con uno* 0 *e poi ci deve essere un* 1*).*

Otteniamo così una sequenza ordinata:

$$M_1,\ M_2,\ M_3, \dots,\ M_n, \dots$$

Dove M_n è la n-esima Macchina di Turing generata da questo algoritmo di enumerazione. Ad ogni macchina corrisponde una funzione:

$$f_1,\ f_2,\ f_3, \dots,\ f_n, \dots$$

Questa è la lista delle funzioni parziali computabili unarie.

2.6.2 L'Insieme K e l'Indecidibilità

Teorema: Definiamo l'insieme K:

$$K = \{a \in \mathbb{N} \mid M_a \text{ termina su } a\} = \{a \in \mathbb{N} \mid f_a(a) \downarrow\}$$

Il quale rispetta le seguenti proprietà:

1. K è semidecidibile.
2. K non è decidibile.

Dimostrazione 1 (Algoritmo di semidecisione per K): Dato $a \in \mathbb{N}$, uso l'algoritmo di enumerazione per ottenere la macchina M_a, ed eseguo M_a su a. Se M_a termina, allora $a \in K$. (Altrimenti non lo so dire se $a \notin K$, il che implica che è semidecidibile).

Dimostrazione 2 : Supponiamo per assurdo che K sia decidibile, cioè abbiamo un algoritmo di decisione per K. Definiamo una funzione $f : \mathbb{N} \to \mathbb{N}$ come segue:

$$f(n) = \begin{cases} f_n(n) + 1 & ,\ n \in K \\ 0 & ,\ n \notin K \end{cases}$$

Osservazione su f: f è computabile. Come lo calcolo in pratica? Uso l'algoritmo di decisione per K: dato $n \in \mathbb{N}$, decido se $n \in K$. Se sì, calcolo $f_n(n) + 1$; se no, scrivo 0.

Dunque f deve comparire nella lista delle funzioni computabili unarie, cioè:

$$\exists \bar{n} \in \mathbb{N} : \quad f = f_{\bar{n}}$$

Calcolo ora $f(\bar{n})$ per valutare i due casi possibili:

- Se $\bar{n} \in K$: per definizione $f(\bar{n}) = f_{\bar{n}}(\bar{n}) + 1$. Ma sapendo che $f = f_{\bar{n}}$, avremmo che $f_{\bar{n}}(\bar{n}) + 1 = f_{\bar{n}}(\bar{n})$. **Assurdo!** (Un numero naturale non può essere uguale al suo successore).
- Se $\bar{n} \notin K$: per definizione di f, avremmo $f(\bar{n}) = 0$. Ma se $\bar{n} \notin K$, significa (per definizione di K) che la macchina non termina, quindi $f_{\bar{n}}(\bar{n}) \uparrow$ (diverge). Essendo $f = f_{\bar{n}}$, avremmo che un valore definito (0) è uguale a qualcosa che non termina. **Assurdo!**

Queste due situazioni incompatibili portano ad un assurdo $\implies$ l'assunto che K sia decidibile è falso.

Problema dell'arresto Data una MDT M e un numero naturale n, determinare se M su n termina.

Teorema (dell'arresto): (Il problema dell'arresto è indecidibile) L'insieme $R = \{(n, m) \in \mathbb{N}^2 \mid M_n$ termina su $m\}$ è semidecidibile, ma non è decidibile.
Dimostrazione intuitiva: Se, per assurdo, R fosse decidibile, allora potrei decidere, in particolare, se $(n, n) \in R$ (per ogni $n \in \mathbb{N}$). Ma tale "sottoproblema" è equivalente al problema di decisione per K, che non è decidibile come abbiamo appena dimostrato.

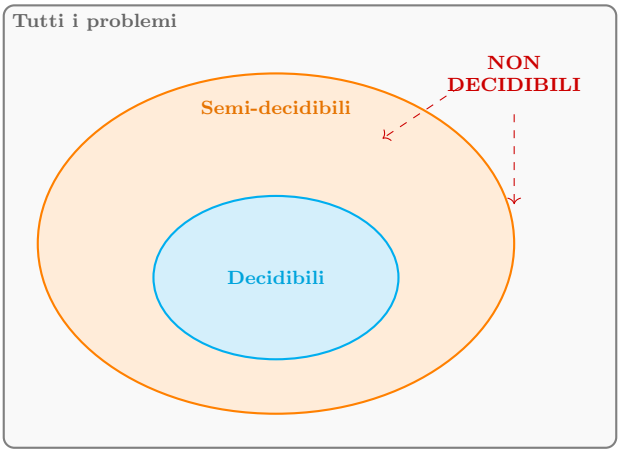
Enunciato equivalente del problema dell'arresto: Data M MDT e w stringa, determinare se M termina su w. Sono equivalenti perché si riduce al fatto che ogni numero naturale si può codificare come stringa su un opportuno alfabeto, e ogni stringa su un alfabeto può essere codificata da un numero naturale.

Funzione di Gödelizzazione Dato $\Sigma = \{a_1, a_2, a_3, \dots\}$, definiamo la funzione $\ddot{g} : \Sigma^* \to \mathbb{N}$:

$$\ddot{g}(a_{i_1} a_{i_2} \dots a_{i_n}) \mapsto P_1^{i_1+1} \cdot P_2^{i_2+1} \dots P_n^{i_n+1}$$

Dove $P = \{P_1, P_2, \dots\} = \{2, 3, 5, \dots\}$ è l'insieme dei numeri primi in ordine crescente.
Proprietà di $\ddot{g}$:

- $\ddot{g}$ è computabile.
- $\ddot{g}$ è iniettiva.
- L'immagine $\ddot{g}(\Sigma^*)$ è decidibile.
- Se $m \in \ddot{g}(\Sigma^*)$, esiste un algoritmo per calcolare $w \in \Sigma^*$ tale che $\ddot{g}(w) = m$.



Definizione: Il linguaggio del problema dell'arresto (o linguaggio dell'arresto) è definito come:

$$L_{HALT} = \{R(M)w \mid M \text{ termina su } w\}$$

Dove $R(M)$ è la codifica binaria di M, concatenata con la stringa w.

Osservazione: Il problema dell'arresto è il problema di decisione per L_{HALT} (data una stringa, verificare se appartiene a L_{HALT}).

Teorema dell'arresto: L_{HALT} è semidecidibile, ma non decidibile.

Proposizione: Il complemento L_{HALT}^C non è semidecidibile. (Le stringhe sintatticamente scorrette o corrette ma dove M non termina).

Dimostrazione: Per assurdo, supponiamo che L^C_{HALT} sia semidecidibile. Sappiamo già che L_{HALT} è semidecidibile. Per il Teorema di Post (se un linguaggio e il suo complemento sono entrambi semidecidibili, allora il linguaggio è decidibile), questo implicherebbe che:

$$\implies L_{HALT} \text{ è decidibile}$$

Assurdo! (Abbiamo appena dimostrato che non lo è).

2.7 Macchina Universale e Riducibilità

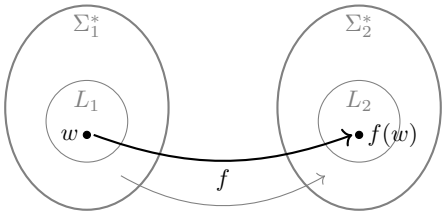
Definizione: Una **MDT Universale** è una Macchina di Turing che, su input $R(M)w$, simula l'esecuzione di M su w.

Proposizione: Esiste una MDT Universale che calcola le funzioni parziali computabili unarie.

Definizione (Riduzione tra linguaggi): Dati L_1, L_2 linguaggi, con $L_1 \subseteq \Sigma_1^*$ e $L_2 \subseteq \Sigma_2^*$, diciamo che L_1 **è riducibile a** L_2 quando:

$$\exists f : \Sigma_1^* \to \Sigma_2^* \quad \text{tale che:}$$

1. f è computabile (esiste una MDT che la calcola).
2. $\forall w \in \Sigma_1^*, \quad w \in L_1 \iff f(w) \in L_2$.



La funzione f si dice **funzione di riduzione** da L_1 a L_2.

Proposizione (Utile per dimostrare la decidibilità):

1. Se abbiamo f riduzione da L_1 a L_2:
   Se L_2 è decidibile $\implies$ L_1 è decidibile.
2. Se abbiamo f riduzione da L_1 a L_2:
   Se L_1 è indecidibile $\implies$ L_2 è indecidibile.

Dimostrazione del punto 1: Siano F la MDT che calcola la funzione f, e M_2 la MDT che decide il linguaggio L_2. Devo costruire una MDT che decide L_1. Se ho in input $w \in \Sigma_1^*$:

- Uso F per calcolare la trasformazione $f(w) \in \Sigma_2^*$.
- Uso M_2 dandogli in pasto $f(w)$ per decidere se $f(w) \in L_2$.

Se M_2 accetta, allora $w \in L_1$. Se M_2 rifiuta, allora $w \notin L_1$. Dunque L_1 è decidibile.

Esempio 1: Siano i linguaggi:

$$L_1 = \{uu \mid u = a^i b^i c^i,\ i \geq 0\} \subseteq \{a, b, c\}^*$$

$$L_2 = \{a^i b^i c^i \mid i \geq 0\} \subseteq \{a, b, c\}^* \quad \text{(Sappiamo già che è decidibile)}$$

Sia M_2 la MDT che decide L_2. Vogliamo costruire una MDT che calcoli una riduzione da L_1 a L_2. Presa in input una stringa $w \in \{a, b, c\}^*$:

- Controlliamo se $w = uu$, per qualche sottostringa $u \in \{a, b, c\}^*$.

- Se <u>non</u> è così, cancello tutto w e scrivo a sul nastro (che sicuramente non sta in L_2, facendola rifiutare).
- Se $w = uu$, cancello la seconda metà di w, ottenendo sul nastro solo u.

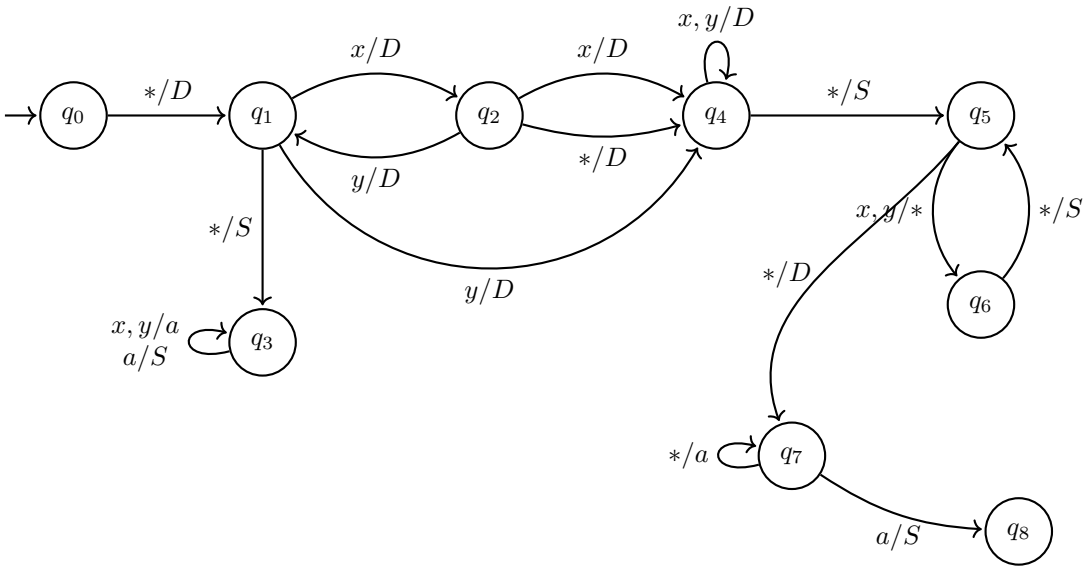
Esempio 2: Siano gli alfabeti $\Sigma_1 = \{x, y\}$ e $\Sigma_2 = \{a\}$.

$$L_1 = \{(xy)^n \mid n \geq 0\} \subseteq \Sigma_1^*$$

$$L_2 = \{a^{2n} \mid n \geq 0\} \subseteq \Sigma_2^*$$

Ecco il diagramma degli stati della MDT che implementa la riduzione da L_1 a L_2 (trasformando le sequenze xy in stringhe di a di lunghezza pari):

```
* x y x y x y x y ...
```



2.7.1 Problema del Nastro Vuoto (BTP)

Definizione (Blank Tape Problem): Data una MDT M, determinare se l'esecuzione di M termina partendo su un nastro completamente vuoto.

Proposizione: Il BTP è **indecidibile**.

Dimostrazione: Descriviamo una riduzione dal Problema dell'Arresto (L_{HALT}, che sappiamo essere indecidibile) al Problema del Nastro Vuoto (L_{BTP}).

$$L_{HALT} = \{R(M)w \mid M \text{ termina su } w\}$$

$$L_{BTP} = \{R(M) \mid M \text{ termina su nastro vuoto (ovvero su stringa } \varepsilon)\}$$

Data una stringa x in input:

- Se x <u>non</u> è della forma $R(M)w$, restituisco 0 (rifiuto subito).
- Altrimenti, cioè se $x = R(M)w$, costruisco una nuova codifica $R(M')$ tale che la nuova macchina M' operi come segue:
  Su un input generico y:
  1. Se $y \neq \varepsilon$, M' si comporta esattamente come M.

2. Se $y = \varepsilon$, M' per prima cosa scrive la stringa w sul nastro, ed esegue M su w.

Osservazione fondamentale:

- Se M termina su $w \implies M'$ termina su ε.
- Se M non termina su $w \implies M'$ non termina su ε.

Dunque vale la doppia implicazione della riduzione:

$$R(M)w \in L_{HALT} \iff R(M') \in L_{BTP}$$

Poiché L_{HALT} è indecidibile e lo abbiamo ridotto a L_{BTP}, per la proposizione precedente anche L_{BTP} deve essere indecidibile.

2.7.2 Proprietà dei Linguaggi Semidecidibili

Restringendoci alla classe dei linguaggi semidecidibili, molti problemi decisionali risultano indecidibili. Definiamo i linguaggi associati ad alcune domande classiche sulle Macchine di Turing:

- **La stringa vuota appartiene al linguaggio?**

$$\{R(M) \mid \varepsilon \in L(M)\}$$

- **Il linguaggio è vuoto?**

$$L_\emptyset = \{R(M) \mid L(M) = \emptyset\}$$

- **Il linguaggio è Σ^*?** (Cioè la macchina accetta tutte le stringhe)

$$L_{\Sigma^*} = \{R(M) \mid L(M) = \Sigma^*\}$$

- **Il linguaggio è regolare?**

$$L_{REG} = \{R(M) \mid L(M) \text{ è regolare}\}$$

In generale, data una generica proprietà P, si definisce il linguaggio:

$$L_P = \{R(M) \mid L(M) \text{ soddisfa la proprietà } P\}$$

Proposizione: L_{Σ^*} NON È DECIDIBILE.
Dimostrazione: Descriviamo una riduzione da L_{HALT} a L_{Σ^*}, ovvero $R(M)w \rightsquigarrow R(M')$. Costruiamo la nuova macchina M' in modo che, su un input generico y, si comporti così:

1. Cancella y dal nastro.
2. Scrive w.
3. Esegue M su w.

Osservazione:

- Se $R(M)w \in L_{HALT}$ (M termina su w) $\implies M'$ termina su qualsiasi y. Quindi M' accetta tutte le stringhe: $R(M') \in L_{\Sigma^*}$.
- Se $R(M)w \notin L_{HALT}$ (M non termina su w) $\implies M'$ non termina su nessun y. Quindi M' non accetta nulla: $R(M') \notin L_{\Sigma^*}$.

2.7.3 Il Teorema di Rice

Definizione (Proprietà Banale): Una proprietà P si dice **BANALE** quando tutti i linguaggi semidecidibili soddisfano P, oppure nessun linguaggio semidecidibile soddisfa P.

TEOREMA DI RICE: Sia P una proprietà **non banale**. Il linguaggio ad essa associato:

$$L_P = \{R(M) \mid L(M) \text{ soddisfa } P\} \text{ NON È DECIDIBILE.}$$

Nota pratica: Questo teorema sancisce che non esiste alcun algoritmo (strategia) capace di analizzare il codice di un programma per determinare se il suo comportamento (il suo linguaggio) possiede una specifica proprietà non banale.

DIMOSTRAZIONE: Sia P una proprietà NON banale.

Caso 1: Supponiamo che $\emptyset$ (il linguaggio vuoto) NON soddisfi P. Poiché P è non banale, esiste almeno un linguaggio semidecidibile L che soddisfa P (quindi $L \neq \emptyset$). Sia M_L la Macchina di Turing che accetta L. Descriviamo una riduzione da L_{HALT} a L_P: $R(M)w \rightsquigarrow R(M')$. **Configurazione Iniziale:** La testina Δ è posizionata all'inizio dell'input y:

$$[\underset{\Delta}{*} \ \dots \ y \ \dots \ * \,]$$

Il comportamento della nuova macchina M' su un input generico y è il seguente:

1. Scrive w a destra di y sul nastro, senza cancellare y. La testina poi torna indietro:

$$[\, * \ \dots \ y \ \dots \ \underset{\Delta}{*} \ \dots \ w \ \dots \,]$$

2. Esegue M su w. La testina opera isolata nella zona di w:

$$[\, * \ \dots \ y \ \dots \ * \ \dots \ \underset{\Delta_M}{w} \ \dots \,]$$

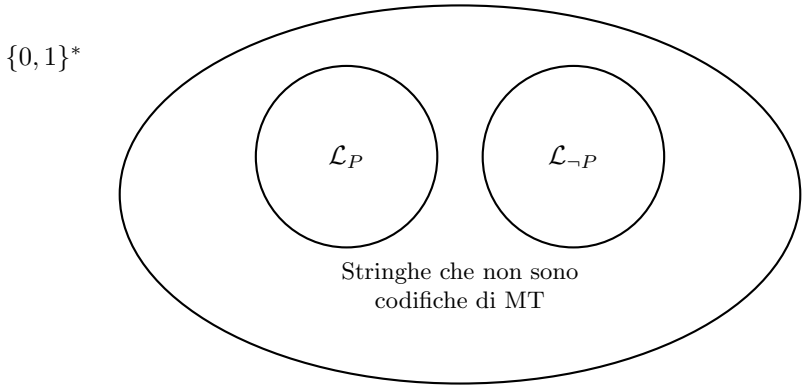
3. Se M termina su w, esegue M_L su y. La testina torna all'inizio sull'input originale:

$$[\, \underset{\Delta_{M_L}}{*} \ \dots \ y \ \dots \ * \,]$$

- Se $R(M)w \in L_{HALT}$ (M termina su w) $\implies$ $L(M') = L(M_L) = L$. Poiché L soddisfa P per ipotesi, allora $R(M') \in L_P$.
- Se $R(M)w \notin L_{HALT}$ (M non termina su w) $\implies$ M' va in loop e non accetta nulla. Quindi $L(M') = \emptyset$. Poiché $\emptyset$ NON soddisfa P, allora $R(M') \notin L_P$.

Quindi $L_{HALT} \leq L_P$, da cui deduciamo che L_P non è decidibile.

Caso 2: Supponiamo che $\emptyset$ SODDISFI P. Allora $\emptyset$ NON soddisfa $\neg P$. Per la dimostrazione precedente (Caso 1), $L_{\neg P}$ **NON è decidibile**.



Dimostrazione per assurdo (P.A.):

Se per assurdo L_P fosse decidibile, allora il suo complementare sarebbe decidibile. Il complementare di L_P è l'unione di $L_{\neg P}$ e della spazzatura. Quindi:

$$L_P \text{ decidibile} \implies L_{\neg P} \cup \{\text{stringhe che non sono codifiche di MT}\} \text{ DECIDIBILE}$$

Ma noi abbiamo un algoritmo che fa un banale controllo sintattico per vedere se una stringa è una codifica valida di una MT. Potremmo quindi filtrare le stringhe e decidere facilmente solo $L_{\neg P}$.

$$\implies L_{\neg P} \text{ DECIDIBILE}$$

ASSURDO! (Perché abbiamo già dimostrato che $L_{\neg P}$ non è decidibile). Di conseguenza, l'ipotesi iniziale cade: L_P **NON è decidibile**. ■

Esempi di Applicazione (e non) del Teorema di Rice

- **Esempio 1 (Applicazione standard):** Data una MT M, determinare se il linguaggio accettato da M contiene più di 100 stringhe.

  $$L_0 = \{R(M) \mid L(M) \text{ ha più di 100 stringhe}\}$$

  Soluzione: La proprietà riguarda $L(M)$ ed è NON BANALE (alcuni linguaggi sono infiniti, altri vuoti o con poche stringhe). Posso usare il Teorema di Rice. $\implies L_0$ **NON È DECIDIBILE.**

- **Esempio 2 (TRAPPOLA STRUTTURALE):** Data M, determinare se l'insieme degli stati di M ha cardinalità maggiore di 50 (supponendo che non ci siano stati inutili). *Soluzione:* **NON SI PUÒ USARE IL TEOREMA DI RICE!** È una proprietà strutturale della macchina, non del suo linguaggio. Il problema è **DECIDIBILE**, perché posso letteralmente contare gli stati scorrendo la codifica della MT (controllando il quarto campo delle transizioni e incrementando un contatore se lo stato è nuovo).

- **Esempio 3 (Proprietà di chiusura):** Data M, determinare se $L(M)$ è chiuso rispetto al ribaltamento della stringa. Ovvero, se $w = w_1 \dots w_n \in L(M)$, allora la sua forma speculare $w^R = w_n \dots w_1 \in L(M)$.

  $$L_0 = \{R(M) \mid L(M) \text{ ha la proprietà indicata}\}$$

  Soluzione: Riguarda il linguaggio ed è una proprietà non banale (alcuni linguaggi sono palindromi, altri no). Posso applicare Rice. $\implies L_0$ **NON È DECIDIBILE.**

Definizione (Macchina Riproducibile): Una MT M si dice **RIPRODUCIBILE** quando $\exists M' \neq M$ tale che $L(M) = L(M')$.

- **Problema 1 (TRAPPOLA DELLA BANALITÀ):** Dato un linguaggio semidecidibile L, determinare se esiste una MT riproducibile M che accetta L.

  $$\{R(M) \mid L(M) \text{ ha la proprietà indicata}\}$$

  Soluzione: **Osservazione Fondamentale:** OGNI singola MT è riproducibile! (Basta prendere M e aggiungerle uno stato vuoto o inutile alla fine per creare una macchina M' diversa ma con lo stesso linguaggio). Poiché tutti i linguaggi possiedono questa proprietà, la proprietà è **BANALE**. Non posso usare Rice. Il problema è **DECIDIBILE** (l'algoritmo che lo decide è un programma banale che restituisce sempre "SÌ").

- **Problema 2 (TRAPPOLA MISTA):** Dato un linguaggio semidecidibile L, determinare se esiste una MDT riproducibile con **meno di 10 stati** che accetta L.

  $$\mathcal{L} = \{R(M) \mid L(M) \text{ è accettato da una MDT riproducibile con meno di 10 stati}\}$$

  Soluzione:

- **Osservazione:** Abbiamo già visto che *ogni* MDT è riproducibile. Quindi la parola "riproducibile" qui è solo un distrattore inserito per confondere.
- La vera proprietà del linguaggio in esame è: *"Essere accettabile da una macchina con meno di 10 stati"*.
- Ci sono linguaggi molto semplici che *sono* accettati da MDT con meno di 10 stati (es. il linguaggio vuoto).
- Ci sono linguaggi complessi che sono accettati *solo* da MDT con più di 10 stati.

Poiché la proprietà divide i linguaggi in due categorie non vuote, **la proprietà è NON BANALE**.
$\Longrightarrow$ Per il Teorema di Rice, $\mathcal{L}$ **NON È DECIDIBILE**.

2.7.4 Equivalenza tra Modelli di Calcolo

Proposizione: Ogni funzione μ-ricorsiva è τ-ricorsiva.

Dimostrazione (Idea): Si procede per induzione strutturale sulla costruzione di una funzione μ-ricorsiva.

1. **Casi Base:** Si dimostra che le funzioni iniziali (Zero, Successore, Proiezione) sono τ-ricorsive (esistono MDT banali che le calcolano).

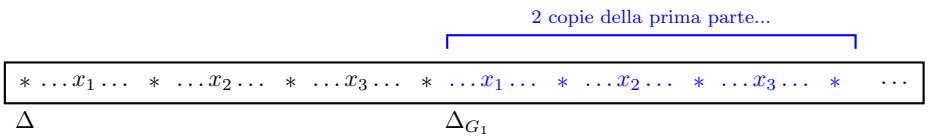
2. **Passo Induttivo (Composizione):**
   Consideriamo ad esempio la composizione $f \circ (g_1, g_2) : \mathbb{N}^3 \to \mathbb{N}$, dove:

   - $g_1, g_2 : \mathbb{N}^3 \to \mathbb{N}$
   - $f : \mathbb{N}^2 \to \mathbb{N}$

   Per ipotesi induttiva, supponiamo di avere già a disposizione:

   - G_1: MDT che calcola g_1
   - G_2: MDT che calcola g_2
   - F: MDT che calcola f

   L'obiettivo è costruire una grande macchina che le assembli. Il primo problema da risolvere è che sia G_1 che G_2 dovranno consumare gli stessi input (x_1, x_2, x_3). Pertanto, la fase preparatoria della nostra nuova macchina consisterà nel copiare l'input originale sul nastro:



Posizionando la testina all'inizio della copia (Δ_{G_1}), possiamo lanciare l'esecuzione della prima sottomacchina preservando i dati originali per la seconda.

3 Teoria della Complessità

Definizione (Funzione di Complessità): Data una Macchina di Turing standard M, definiamo la sua funzione di complessità temporale $tc_M : \mathbb{N} \to \mathbb{N}$ come:

$tc_M(n) =$ numero di transizioni eseguite da M su una stringa di lunghezza n nel **caso peggiore**

3.1 Notazioni Asintotiche

Per confrontare l'efficienza degli algoritmi, ignoriamo le costanti e i termini di grado inferiore, concentrandoci sull'ordine di grandezza al crescere di n. Siano $f, g : \mathbb{N} \to \mathbb{N}$:

- **Limite Superiore ($\mathcal{O}$ Grande):** $f \in \mathcal{O}(g)$ quando $\exists c > 0, \exists n_0 \in \mathbb{N}$ tale che $\forall n \geq n_0$,

$$\frac{f(n)}{g(n)} \leq c$$

- **Limite Inferiore (Ω):** $f \in \Omega(g)$ quando $\exists d > 0, \exists n_0 \in \mathbb{N}$ tale che $\forall n \geq n_0$,

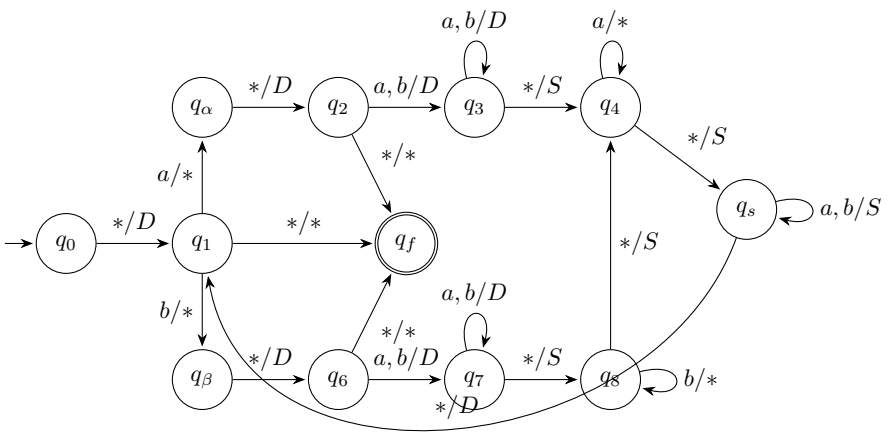
$$\frac{f(n)}{g(n)} \geq d$$

- **Limite Esatto (Θ):** $f \in \Theta(g)$ quando $f \in \mathcal{O}(g)$ e $f \in \Omega(g)$. Cioè $\exists c, d > 0$ tali che $\forall n \geq n_0$:

$$c \leq \frac{f(n)}{g(n)} \leq d$$

- **o piccolo:** $f \in o(g)$ quando $\lim_{n \to +\infty} \frac{f(n)}{g(n)} = 0$. (Nota: quando $f \in o(g) \implies f \in \mathcal{O}(g)$).
- **Equivalenza Asintotica ($\sim$):** $f \sim g$ quando $\lim_{n \to +\infty} \frac{f(n)}{g(n)} = 1$. (Nota: quando $f \sim g \implies f \in \Theta(g)$).

Esempio: $L \subseteq \{a, b\}^*$
L è il linguaggio delle palindrome su $\{a, b\}$. Esempio stringa sul nastro: $*abbabba$



Come funziona (in breve):

La macchina verifica se la stringa è palindroma smangiandola dagli estremi verso il centro:

1. **Lettura e Memorizzazione:** In q_1 legge il primo carattere, lo cancella (scrive '*') e "memorizza" cosa ha letto prendendo la strada in alto (se era 'a') o in basso (se era 'b').
2. **Corsa a destra:** Scorre l'intera stringa (q_3 o q_7) finché non sbatte contro il '*' che delimita la fine.
3. **Verifica finale:** Fa un passo indietro. Se si trova sul ramo alto (q_4), si aspetta di trovare una 'a'; se si trova sul ramo basso (q_8) si aspetta una 'b'. Se il carattere corrisponde, lo cancella (sostituendolo con '*').
4. **Ritorno:** Sfrutta lo stato q_s per riavvolgere il nastro tutto a sinistra, fino a trovare il primo '*'. A quel punto fa un passo a destra e ricomincia il ciclo da q_1.
5. **Accettazione:** Se il nastro si svuota (cioè in q_1, q_2 o q_6 legge direttamente '*' invece di una lettera), la parola era palindroma e va in q_f.

Calcolo della Complessità nel Caso Peggiore (Accettazione)

Sia n la lunghezza della stringa di input.

Caso pari ($n = 2k$):

$$\left.\begin{array}{l} 1+2+(2k-1)+ \\ 1+2+(2k-2)+ \\ \vdots \\ 1+2+1+ \\ 1+2+0+ \\ 2 \end{array}\right| = 2 + \sum_{i=0}^{2k-1}(3+i) = 2 + \sum_{i=0}^{2k-1} 3 + \sum_{i=0}^{2k-1} i = 2 + 3(2k) + \frac{2k(2k-1)}{2}$$

$$= \mathbf{\frac{n(n-1)}{2} + 3n + 2} \implies \Theta(n^2)$$

Caso dispari ($n = 2k-1$):

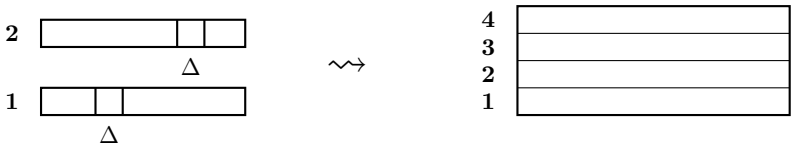
$$\left.\begin{array}{l} 1+2+(2k-2)+ \\ 1+2+(2k-3)+ \\ \vdots \\ 1+2+2+ \\ 1+2+1+ \\ 4 \end{array}\right| = 4 + \sum_{i=1}^{2k-2}(3+i) = 4 + 3(2k-1) + \sum_{i=1}^{2k-2} i = 4 + 3(2k-1) + \frac{(2k-1)(2k-2)}{2}$$

$$= \mathbf{4 + 3n + \frac{n(n-1)}{2}} \implies \Theta(n^2)$$

Costo Temporale delle Varianti di MDT

Prop: Se M è una MDT a k tracce che accetta L con $tc_M(n) = f(n) \implies \exists M'$ MDT standard che accetta L tale che $tc_{M'}(n) = f(n)$.

Prop: Se M è una MDT a k nastri ($k > 1$) che accetta L con $tc_M(n) = f(n) \implies \exists M'$ MDT standard che accetta L tale che $tc_{M'}(n) = \mathcal{O}(f(n)^2)$.



Analisi dei costi per la t-esima transizione di M:
Al passo t, le testine virtuali si sono potute allontanare al massimo di t celle dall'inizio. Per simulare un singolo passo, la testina della nuova macchina deve fare una scansione andata e ritorno per leggere i dati e un'altra per aggiornarli:

$$\text{Costo } t\text{-esima transizione di } M \qquad 2t \cdot k + 2t \cdot k = 4tk$$

Stima del limite superiore del numero di transizioni eseguite da M' su input di lunghezza n nel caso peggiore:

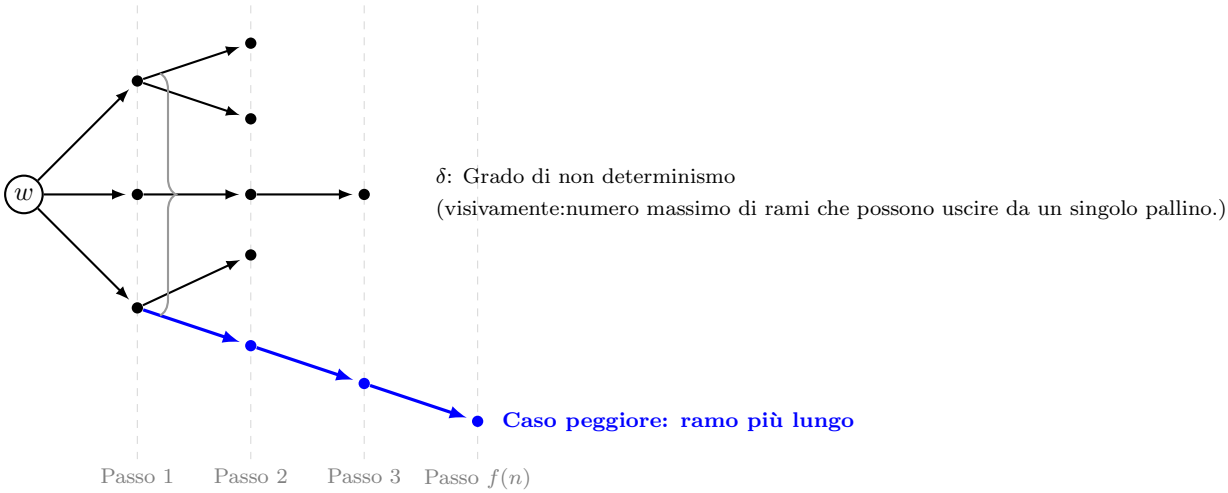
$$\sum_{t=1}^{f(n)} 4tk = 4k \sum_{t=1}^{f(n)} t = 4k \cdot \frac{f(n) \cdot (f(n)+1)}{2} = \mathbf{\Theta(f(n)^2)}$$

3.2 Complessità in tempo su MDT Non Deterministiche

Definizione: Sia M una MDT Non Deterministica. Definiamo la sua complessità in tempo come:

$tc_M(n) =$ numero di transizioni eseguite da M su una stringa di lunghezza n nel **caso peggiore**

Nota bene: Poiché M è non deterministica, per una singola stringa w esistono *molteplici* percorsi di computazione (un albero di scelte). Il "caso peggiore" si calcola **scegliendo la computazione che massimizza il numero di transizioni** (il ramo più lungo dell'albero).



Proposizione (Simulazione Deterministica): Sia L un linguaggio accettato da una MDT non deterministica M, e sia $tc_M(n) = f(n)$. $\implies \exists M'$ MDT **deterministica** che accetta L tale che:

$$tc_{M'}(n) = \mathcal{O}\Big(f(n) \cdot \delta^{f(n)}\Big)$$

*(La complessità diventa **esponenziale** rispetto a M)*

Dimostrazione (Idea della simulazione):
Costruiamo una MDT **deterministica** M' a 3 nastri che sia equivalente a M. I tre nastri vengono impiegati nel seguente modo:

- **Nastro 1:** Contiene l'input originale (sola lettura).

- **Nastro 2:** Viene utilizzato per la simulazione effettiva delle operazioni di M.
- **Nastro 3:** Contiene le computazioni di M, ovvero le sequenze delle scelte non deterministiche, rappresentate come vettori $(u_1, u_2, \ldots, u_r)$ dove $1 \leq u_i \leq \delta$.

La strategia di M' è sistematica: prima scrive tutte le possibili sequenze di computazione sul Nastro 3 (una dopo l'altra), e poi le esegue materialmente sul Nastro 2. Per calcolare il costo temporale totale dell'algoritmo deterministico, la logica da seguire è: *contare quanto serve per generare le computazioni e moltiplicarlo per quanto si impiega a eseguirle.*

1. **Numero totale di sequenze generate:** Il numero di computazioni (sequenze) distinte di M, considerando una lunghezza massima $f(n)$ su un input di lunghezza n, è delimitato superiormente da $\delta^{\mathbf{f(n)}}$.
2. **Costo di una singola simulazione:** Il numero di transizioni eseguite da M per simulare una singola computazione su una stringa di lunghezza n è al massimo $\mathbf{f(n)}$.

Il costo totale nel caso peggiore (il limite superiore) è dato dal prodotto di questi due fattori. Ne consegue che la complessità in tempo della macchina deterministica M' è:

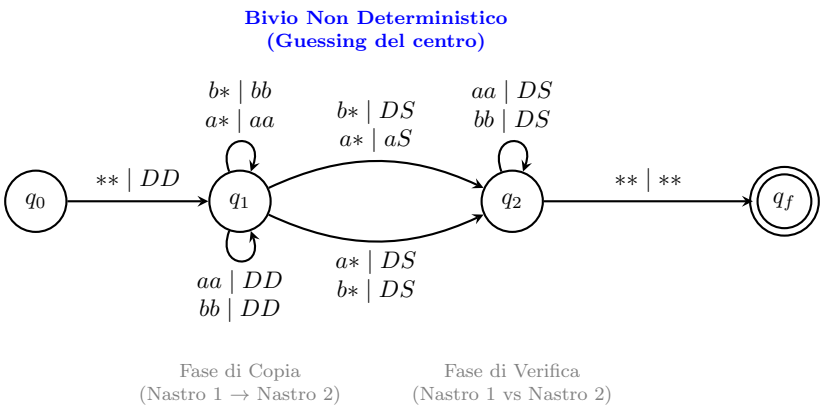
$$tc_{M'}(n) = \mathcal{O}\Big(f(n) \cdot \delta^{f(n)}\Big)$$

Osservazione: Questa dimostrazione evidenzia formalmente come l'eliminazione del non-determinismo comporti un'inevitabile esplosione **esponenziale** dei tempi di calcolo.

Esempio: Palindroma Binaria su MDT Non Deterministica

Vogliamo riconoscere il linguaggio delle palindrome binarie sfruttando il non-determinismo. Con una MDT Non Deterministica a 2 nastri cambiamo strategia rispetto all'MDT deterministica, e "tiriamo a indovinare" le scelte corrette diramando la computazione ("guessing"). La macchina procede così:

1. **Copia:** Scorre il Nastro 1 verso destra e copia i caratteri sul Nastro 2.
2. **Il Guessing (La Scelta):** Ad ogni passo, la macchina compie una scelta non deterministica. Può decidere di continuare a copiare, oppure "indovinare" di aver raggiunto l'esatto centro della stringa e cambiare stato.
3. **Verifica:** Se dichiara di essere al centro, smette di copiare. Inizia a confrontare il resto della parola sul Nastro 1 (procedendo verso destra) con la copia sul Nastro 2 (procedendo a ritroso verso sinistra).



Complessità nel caso peggiore (Rifiuto): Il caso peggiore in questa simulazione si verifica quando la computazione ritarda il più possibile il fallimento. Questo accade se la macchina "indovina" erroneamente che il centro si trovi alla fine esatta della stringa: copierà tutti gli n caratteri della stringa prima

di accorgersi dell'errore (o di terminare l'input). Il numero di passi sarà 1 (per la partenza) $+n$ (per copiare tutta la stringa) $+n$ (per tornare indietro se ipoteticamente verificasse). In ogni caso:

$$tc_M(n) = 1 + 2n = \mathcal{O}(n)$$

Nota metodologica: La complessità è crollata da $\mathcal{O}(n^2)$ a $\mathcal{O}(n)$. È tuttavia importante sottolineare che non ha senso confrontare direttamente le prestazioni dell'algoritmo deterministico a un nastro con questo non deterministico, perché abbiamo cambiato strategia computazionale.

3.3 Le Classi P e NP

- **La Classe $\mathcal{P}$:** Linguaggi decidibili in tempo polinomiale da una macchina **deterministica**.

$$\mathcal{P} = \{L \mid \exists M \text{ MDT det. che accetta } L \text{ t.c. } tc_M(n) = \mathcal{O}(n^r), \exists r \in \mathbb{N}\}$$

- **La Classe $\mathcal{NP}$:** Linguaggi decidibili in tempo polinomiale da una macchina **non deterministica**.

$$\mathcal{NP} = \{L \mid \exists M \text{ MDT non det. che accetta } L \text{ t.c. } tc_M(n) = \mathcal{O}(n^r), \exists r \in \mathbb{N}\}$$

Osservazione: $\mathcal{P} \subseteq \mathcal{NP}$ (una MDT deterministica è una MDT non deterministica con $\delta = 1$). Non sappiamo dire se $\mathcal{NP} \subseteq \mathcal{P}$. È un **PROBLEMA APERTO** (il più importante dell'informatica).

3.3.1 Problema del circuito hamiltoniano (HAM)

Definizione: Dato un grafo orientato $G = (V, E)$ (dove V sono i vertici ed E gli archi), un **circuito hamiltoniano** in G è una sequenza $(x_1, x_2, \ldots, x_{n-1}, x_n, x_1)$ di vertici tale che:

- $\forall i,\ (x_i, x_{i+1}) \in E$
- $(x_n, x_1) \in E$
- $V = \{x_1, \ldots, x_n\}$

In parole povere: è un ciclo che passa una e una sola volta per tutti i vertici del grafo.

Problema: Dato un grafo orientato G, determinare se G possiede un circuito hamiltoniano.

Codifica in una Macchina di Turing (M):
Consideriamo $G = (V, E)$ con $V = \{1, 2, \ldots, n\}$.

- **Codifica dei vertici:** Si usa la codifica binaria (indicheremo la codifica di x_i con $\underline{x_i}$).
- **Codifica degli archi:** Un arco (x_i, x_j) viene codificato come $\underline{x_i}\#\underline{x_j}$.
- **Codifica del grafo:** Il grafo sul nastro è codificato come la lista di tutti i suoi archi, seguita dal numero totale di vertici n.

Esempio di come appare il grafo codificato sul nastro:

$$\ldots \underline{x_i}\#\underline{x_j}\#\#\underline{x_k}\#\underline{x_h}\#\#\ldots\#\#\#\underline{n}$$

(I simboli # fungono da separatori tra i vari elementi codificati).

Setup dei Nastri (MDT a 4 nastri):

- **Nastro 4:** [$*$ 1 $*$ n ... $*$ n $*$ 1 *(Stringa terminatrice per il controllo)*
- **Nastro 3:** [$*$ 1 $*$ i_1 *(Memoria per i vertici che hanno già passato i controlli)*
- **Nastro 2:** [$*$ 1 $*$ i_1 $*$ i_2 ... $*$ i_{n-1} $*$ 1 *(Sequenza attualmente in esplorazione)*

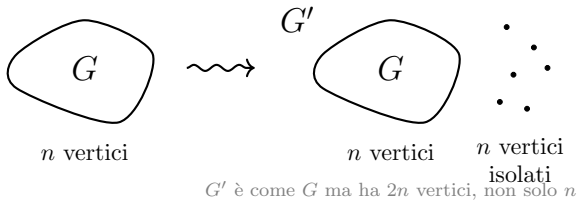
- **Nastro 1:** `[ GRAFO IN INPUT ]`

Procedura dell'algoritmo deterministico:

1. Scrivo sul **Nastro 4** la sequenza di terminazione $*1 * n \cdots * n * 1$ (di lunghezza $n+1$).
2. Sul **Nastro 2** genero, una dopo l'altra in ordine lessicografico, le stringhe di lunghezza $n+1$ composte da vertici di V che iniziano e finiscono con 1.
3. Confronto la stringa generata sul Nastro 2 col contenuto del Nastro 4:
   - Se sono uguali $\implies$ **RIFIUTO** (ho esaurito tutte le combinazioni possibili).
   - Altrimenti, avvio il controllo sulla stringa generata sul Nastro 2.
4. Scorrendo la sequenza, per ogni vertice i_j verifico due condizioni:
   - Controllo che i_j non compaia tra gli elementi i_k con $k < j$ (leggendo i vertici già validati e scritti sul Nastro 3).
   - Controllo che l'arco $(i_{j-1}, i_j) \in E$ (cercandolo nel Nastro 1).
5. Se **entrambi** i controlli sono passati, scrivo i_j sul Nastro 3 e procedo col vertice successivo. Altrimenti, abortisco la verifica di questa stringa e prendo la **prossima sequenza** sul Nastro 2.

Complessità nel caso peggiore (Rifiuto della stringa):
Il caso pessimo per questo algoritmo si verifica quando il grafo in input **non possiede** alcun circuito hamiltoniano. In questa situazione, la macchina non troverà mai una sequenza valida e sarà costretta a generare e verificare *tutte* le possibili combinazioni fino a raggiungere la stringa limite sul Nastro 4, prima di potersi fermare e rifiutare l'input. Il numero totale di sequenze da generare e testare sul Nastro 2 è circa $\mathbf{n^{n-1}}$. Di conseguenza, il tempo di calcolo nel caso peggiore esplode, confermando una complessità **fortemente esponenziale**.

Nota di implementazione (Padding): Per garantire che l'algoritmo brute-force abbia sempre lo spazio su nastro necessario per testare sequenze sufficientemente lunghe, si applica una trasformazione al dato in ingresso. Si mappa il grafo originale G in un nuovo grafo G' aggiungendo un numero di vertici isolati pari a n.



Lavorando sul grafo espanso G' a $2n$ vertici, le sequenze da generare e testare nel caso peggiore salgono a $\mathbf{(2n)^{2n-1}}$.

Osservazione Fondamentale su HAM e la classe $\mathcal{P}$:
Abbiamo appena dimostrato che questo specifico algoritmo deterministico per risolvere HAM impiega un tempo esponenziale $\mathcal{O}((2n)^{2n})$. Tuttavia, **da questo non possiamo assolutamente dedurre che HAM $\notin \mathcal{P}$.**

MDT Non Deterministica che risolve HAM

L'idea alla base di questo algoritmo è sfruttare il non-determinismo per non controllare tutte le combinazioni, ma generare un'unica computazione che "indovina" la soluzione corretta.

Dato il grafo $G = (V, E)$, con $|V| = n$ e $|E| = k$.

Setup dei Nastri:

- **Nastro 3:** `[` $*\ 1\ *\ i_1\ *\ \ldots\ *\ i_j\ *$ *(Controllo)*
- **Nastro 2:** `[` $*\ 1\ *\ i_1\ *\ \ldots\ *\ i_{j+1}\ *\ \ldots\ *\ i_{n-1}\ *\ 1$ *(Generazione della sequenza di vertici)*
- **Nastro 1:** `[ ... CODIFICA DEL GRAFO ...` *(Input)*

Analisi della lunghezza dell'input:
La codifica del grafo sul Nastro 1 appare strutturata nel seguente modo:

$$\cdots\#\#\underbrace{\underline{i}}_{\log n}\#\underbrace{\underline{j}}_{\log n}\#\#\cdots\underbrace{\#\#\#}_{\text{separatori}}\underbrace{\underline{n}}_{\log n}$$

Considerando l'intero blocco degli archi, la lunghezza complessiva dell'input è limitata da:

$$\log n + k \log n \leq (k+1)\log k \leq k^2$$

Comportamento della MDT:

1. **Termino rifiutando** se $n > k$.
2. **Genero non deterministicamente** sul Nastro 2 una sequenza di vertici "candidato" circuito hamiltoniano.
3. **Controllo** (usando il Nastro 3) se la sequenza generata è davvero un circuito hamiltoniano.

Caso peggiore: Accettazione
Osservazione: In caso di accettazione, si ha sicuramente $k \geq n$. Pertanto, possiamo limitarci ad esprimere la complessità in tempo della MDT in funzione solamente di k.

Calcolo della complessità in tempo:

- **Controllo iniziale:** $\mathcal{O}(k \log n)$
- **Generazione della sequenza:** $\mathcal{O}(n \log n)$
  (generando $n+1$ vertici, impiegando $2\log n$ transizioni per vertice)
- $\forall$ **iterazione del ciclo di controllo:**
  - Controllo che il nuovo vertice non sia già comparso (scorrendo la lista): $\mathcal{O}(n \log n)$
  - Verifico che ci sia un arco dal vertice precedente al nuovo vertice: $\mathcal{O}(k \log n)$
  - Riposiziono la testina del Nastro 3: $\mathcal{O}(n \log n)$

  Ciascuna iterazione costa al massimo $\mathcal{O}(k \log n)$ transizioni. Poiché ci sono circa n iterazioni, la complessità del ciclo di controllo ammonta a:
- **Complessità del ciclo:** $\mathcal{O}(nk \log n)$

Sommando tutte le fasi, la complessità totale risulta:

$$\begin{aligned}
&\mathcal{O}(k \log n) + \mathcal{O}(n \log n) + \mathcal{O}(nk \log n) \\
&\leq \mathcal{O}(k \log k) + \mathcal{O}(k \log k) + \mathcal{O}(k^2 \log k) \\
&\leq \mathcal{O}(k^2 \log k) \\
&\leq \mathcal{O}(k^3) \quad \longleftarrow \text{Tempo Polinomiale!}
\end{aligned}$$

Pertanto, **HAM** $\in$ **NP**.

Domanda fondamentale: Costruire una soluzione è più costoso della verifica della soluzione? ($\mathbf{P} \stackrel{?}{=} \mathbf{NP}$)

3.3.2 Riducibilità Polinomiale

Siano L_1, L_2 linguaggi, con $L_1 \subseteq \Sigma_1^*$ e $L_2 \subseteq \Sigma_2^*$.

Definizione: L_1 è **polinomialmente riducibile** a L_2 quando:

1. L_1 è riducibile a L_2, cioè $\exists f : \Sigma_1^* \to \Sigma_2^*$ tale che:
   - $\forall w \in \Sigma_1^*, \quad w \in L_1 \iff f(w) \in L_2$
   - f è computabile
2. f è computabile in tempo polinomiale.

Proposizione: Siano $L_1 \subseteq \Sigma_1^*$ e $L_2 \subseteq \Sigma_2^*$. Se L_1 è polinomialmente riducibile a L_2 e $L_2 \in P \implies \mathbf{L_1} \in \mathbf{P}$.

Dimostrazione:
Costruiamo una MDT N che decide L_1. Su input $w \in \Sigma_1^*$:

1. Calcolo $f(w)$ (usando la macchina F).
2. Decido se $f(w) \in L_2$ (usando la macchina M).

Sappiamo che: $\begin{cases} f \text{ è la riduzione polinomiale da } L_1 \text{ a } L_2 \ (f : \Sigma_1^* \to \Sigma_2^*) \\ F \text{ è la MDT che calcola } f \text{ con } tc_F(n) \in \mathcal{O}(n^r) \\ M \text{ è la MDT deterministica che decide } L_2, \text{ con } tc_M(n) \in \mathcal{O}(n^s) \end{cases}$
con r e s interi positivi (essendo f e M polinomiali).

Calcoliamo la **complessità in tempo** della macchina N:

$$\begin{aligned} tc_N(n) &= tc_F(n) + tc_M(n^r) \\ &\in \mathcal{O}(n^r) + \mathcal{O}((n^r)^s) \\ &= \mathcal{O}(n^r) + \mathcal{O}(n^{rs}) \\ &= \mathcal{O}(n^{rs}) \end{aligned}$$

Essendo limitata da un polinomio, segue che la macchina N opera in tempo polinomiale, quindi $L_1 \in P$.

3.3.3 Classi NP-difficile e NP-completo

Definizione: Un linguaggio L si dice **NP-difficile** quando:

$$\forall Q \in NP, \quad Q \text{ è polinomialmente riducibile a } L$$

Definizione: Un linguaggio L si dice **NP-completo** quando soddisfa le seguenti condizioni:

1. L è NP-difficile
2. $L \in NP$

Si definisce $\mathbf{NPC} = \{L \mid L \text{ è NP-completo}\}$.
Vale la relazione $\mathbf{NPC} \subseteq \mathbf{NP}$ *(è la classe dei problemi più "difficili" contenuta in NP).*

3.3.4 Relazione tra P, NP e NPC

Proposizione: Supponiamo che $\exists L$ tale che $L \in NPC$ e $L \in P$, allora $\mathbf{P} = \mathbf{NP}$.
In forma logica: $\Big((\exists L)(L \in NPC \text{ e } L \in P)\Big) \implies P = NP$.

Dimostrazione:
Sappiamo già che $P \subseteq NP$. Vogliamo dimostrare che $NP \subseteq P$.
Sia $Q \in NP$. Poiché L è NP-difficile per ipotesi, $\exists f$ riducibilità polinomiale da Q a L.
Poiché $L \in P$, $\exists M$ MDT deterministica polinomiale che accetta L.

Costruiamo la MDT che accetta Q:

$$\boxed{\textbf{Deterministica Polinomiale}}$$

Su input w:

- Calcolo $f(w)$ *(eseguito in tempo polinomiale)*
- Decido se $f(w) \in L$ *(uso M, eseguito in tempo polinomiale)*

$$\Downarrow$$
$$\mathbf{Q} \in \mathbf{P}$$

Avendo dimostrato che per un qualunque problema $Q \in NP$ si ha che $Q \in P$, deduciamo che $NP \subseteq P$.
Concludiamo quindi che $\mathbf{P} = \mathbf{NP}$.

3.4 Il Problema SAT (Satisfiability)

3.4.1 Rappresentazione delle Istanze di un Problema

Siano date le istanze di un problema $\{p_1, p_2, \dots, p_i, \dots\}$ e due loro possibili rappresentazioni:

$$rep_1 : \{p_1, p_2, \dots, p_i, \dots\} \longrightarrow \Sigma_1^*$$
$$rep_2 : \{p_1, p_2, \dots, p_i, \dots\} \longrightarrow \Sigma_2^*$$

Definizione: rep_1 è **polinomialmente trasformabile** in rep_2 quando $\exists t : \Sigma_1^* \to \Sigma_2^*$ tale che:

i) $\forall i, \quad t(rep_1(p_i)) = rep_2(p_i)$

ii) $\forall w \in \Sigma_1^*, \quad w \notin \mathrm{Im}(rep_1) \implies t(w) \notin \mathrm{Im}(rep_2)$

iii) t è computabile in tempo polinomiale

Osservazione: Se rep_1 è polinomialmente trasformabile in rep_2, allora la lunghezza di $rep_2(p_i)$ (che è uguale a $t(rep_1(p_i))$) è al più polinomiale nella lunghezza di $rep_1(p_i)$. Pertanto, se un problema sta in $\mathcal{P}$ usando rep_2, allora il problema sta in $\mathcal{P}$ anche usando rep_1.

$$\boxed{\textbf{Attenzione al caso delle rappresentazioni binaria e unaria di un numero naturale!}}$$

La trasformazione da binario a unario **non è polinomiale** (è esponenziale)! Può accadere che un problema stia in $\mathcal{P}$ con la rappresentazione unaria, ma non stia in $\mathcal{P}$ con la rappresentazione binaria.

3.4.2 Polinomi Booleani

Definizione: Dati $x_1, x_2, \dots, x_m$ indeterminate, definiamo il **Polinomio Booleano** in $x_1, x_2, \dots, x_m$.
L'insieme $PB(x_1, \dots, x_m)$ dei polinomi booleani è definito induttivamente:

- $0, 1 \in PB(x_1, \dots, x_m)$
- $\forall i \leq m, \quad x_i \in PB(x_1, \dots, x_m)$

- $p, q \in PB(x_1, \ldots, x_m) \implies (p \vee q), (p \wedge q), p' \in PB(x_1, \ldots, x_m)$
- Nient'altro è un polinomio booleano.

Esempi di appartenenza a $PB(x, y, z)$: $0,\ y,\ x \vee y,\ (x' \wedge z)' \vee (x \vee y)'$

Esempio di valutazione:
Dato il polinomio $(x' \wedge z)' \vee (x \vee y)'$, valutiamolo con l'assegnamento $x = 0, y = 1, z = 0$:

$$(0' \wedge 0)' \vee (0 \vee 1)' \implies (1 \wedge 0)' \vee (1)' \implies (0)' \vee 0 \implies 1 \vee 0 \implies \mathbf{1}$$

Tabella di verità per $x \wedge y$:

x	y	$x \wedge y$
0	0	0
0	1	0
1	0	0
1	1	1

Definizione: Due polinomi booleani p, q sono **equivalenti** ($p \equiv q$) quando, $\forall t$ assegnamento di valore booleano alle variabili, si ha $t(p) = t(q)$.

Definizioni su $PB(x_1, \ldots, x_m)$:

- **Letterale:** Variabile o negazione di variabile (es. x_i o x_i').
- **Clausola:** Disgiunzione di letterali (es. $x_2 \vee x_4' \vee x_1' \vee x_8$).
- **Polinomio Booleano in Forma Normale Congiuntiva (CNF):** Congiunzione di clausole.
  Esempio: $(x_1 \vee x_3') \wedge (x_2 \vee x_3 \vee x_5') \wedge x_1'$

Definizione: Un polinomio booleano p si dice **soddisfacibile** quando $\exists t$ assegnamento tale che $t(p) = 1$.
Osservazione: Un assegnamento t per le variabili $\{x_1, \ldots, x_m\}$ è una funzione $t : \{x_1, \ldots, x_m\} \to \{0, 1\}$.

3.4.3 Definizione del Problema SAT

Definizione (SAT): Dato un polinomio booleano p in CNF, determinare se p è soddisfacibile.

Codifica di un Polinomio Booleano in CNF:
Considerando le variabili $\{x_1, \ldots, x_m\}$ e l'alfabeto $\Sigma_{SAT} = \{0, 1, \#, \vee, \wedge\}$:

- **Variabili:** $x_i \rightsquigarrow \underline{i}$ *(dove $\underline{i}$ è la codifica binaria di i)*
- **Letterali:**
  - Variabile affermata: $x_i \rightsquigarrow \underline{i}\#1$
  - Variabile negata: $x_i' \rightsquigarrow \underline{i}\#0$

Esempio di codifica:

$$(x_1 \vee x_2') \wedge (x_1' \vee x_3) \quad \rightsquigarrow \quad 1\#1 \vee 10\#0 \wedge 1\#0 \vee 11\#1$$

Codifica di un'istanza di SAT completa:
Lista delle variabili ## Codifica del Polinomio Booleano
Esempio: $1\#10\#11\#\#\ldots$

Proposizione: SAT $\in$ NP
Dimostrazione: Costruiamo una MDT Non Deterministica che risolve SAT.
Idea: Genero non deterministicamente un assegnamento e controllo se soddisfa il polinomio.

Setup dei Nastri (MDT a 2 nastri):

- **Nastro 2:** `[` ... $\underline{i}$#0##...##$\underline{j}$#1##... *(Generazione assegnamento per $\{x_1, \ldots, x_m\}$)*
- **Nastro 1:** `[ ... istanza var ## cod pol. ...` *(Input)*

Logica di verifica:
Per ogni letterale letto sull'input (Nastro 1), controllo il valore binario assegnato a quel letterale sul Nastro 2:

1. **Se coincidono** (il letterale valuta a 1): La clausola corrente è soddisfatta! Passo ad analizzare la clausola successiva. Se la clausola era l'ultima, **accetto**.
2. **Se sono diversi** (il letterale valuta a 0): Passo ad analizzare il letterale successivo della stessa clausola. Se il letterale era l'ultimo della sua clausola (e nessuno l'ha soddisfatta), **rifiuto**.

Calcolo della Complessità nel Caso Peggiore (Accettazione):
Siano:

- n = numero di variabili
- k = numero totale di letterali nel polinomio

La lunghezza totale dell'input è limitata da:

$$\underbrace{n \cdot \log n}_{\text{codifica lista variabili}} + \underbrace{k \cdot \log n}_{\text{codifica dei letterali}} = (\mathbf{n} + \mathbf{k}) \log \mathbf{n}$$

Stima del numero di transizioni:

$$\begin{aligned} tc_M &\approx \underbrace{n \log n}_{\text{generazione}} + \underbrace{k \cdot n \log n}_{\text{verifica}} \\ &\leq n^2 + kn^2 \\ &\leq \Big((n+k)\log n\Big)^2 + \Big((n+k)\log n\Big)^3 \end{aligned}$$

Essendo il numero di transizioni limitato superiormente da un polinomio il cui termine dominante è cubico, concludiamo che l'algoritmo termina in tempo polinomiale. Pertanto, **SAT $\in$ NP**.

3.5 Teorema di Cook: SAT è NP-difficile

Dimostrazione:
Sia $L \in \mathcal{NP}$.
Sia M una Macchina di Turing non deterministica polinomiale che accetta L.
Sia $tc_M(n) = p(n)$ (un polinomio in n che rappresenta il numero massimo di transizioni di M nel caso peggiore).

Vogliamo costruire una riducibilità polinomiale da L a SAT. Dobbiamo cioè trovare una funzione:

$$f : \Sigma_L^* \longrightarrow \Sigma_{\text{SAT}}^*$$

Tale che f sia computabile in tempo polinomiale, e che valga la seguente proprietà:

$$\forall w \in \Sigma_L^*, \quad w \in L \iff f(w) \text{ è un polinomio booleano soddisfacibile}$$

Ricordando che $w \in L \iff M$ accetta w.

Osservazione: Non è restrittivo supporre che, $\forall w$ di lunghezza n, ogni computazione di M su w esegua **esattamente** $p(n)$ transizioni (basta "allungare" artificialmente le computazioni che terminerebbero prima).

Parametri della Macchina di Turing

Siano dati:

- Q = insieme degli stati di $M = \{q_1, q_2, \ldots, q_s\} \implies |Q| = s$
- Σ = alfabeto di lavoro di $M = \{a_1, a_2, \ldots, a_r\} \implies |\Sigma| = r$

Definizione delle Variabili Booleane

Per descrivere ogni istante della computazione, utilizziamo 3 tipi di variabili booleane:

1. $\mathbf{S(u,t)}$:
   Se $S(u,t) = 1$ significa che all'istante t la macchina M si trova nello stato q_u.
   Se $S(u,t) = 0 \implies$ la macchina non è in quello stato.

$$t \in \{0, 1, \ldots, p(n)\} \quad \text{(istante di tempo)}$$
$$u \in \{1, 2, \ldots, s\} \quad \text{(indice dello stato)}$$

2. $\mathbf{C(i,j,t)}$:
   Se $C(i,j,t) = 1$ significa che all'istante t nella cella i-esima del nastro c'è il simbolo a_j.

$$t \in \{0, 1, \ldots, p(n)\}$$
$$i \in \{1, 2, \ldots, p(n)+1\} \quad \text{(indice della cella)}$$
$$j \in \{1, 2, \ldots, r\} \quad \text{(indice del simbolo)}$$

3. $\mathbf{L(i,t)}$:
   Se $L(i,t) = 1$ significa che all'istante t la testina di M si trova sulla cella i.

$$t \in \{0, 1, \ldots, p(n)\}$$
$$i \in \{1, 2, \ldots, p(n)+1\}$$

Le 7 Condizioni di Validità della Computazione

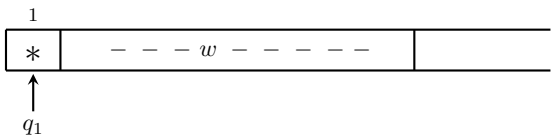
Affinché i valori di queste variabili descrivano una computazione sensata e corretta di M, devono valere le seguenti regole:

1. $\forall t,\ \exists! u : S(u,t) = 1$
   (Ad ogni istante c'è un unico stato in cui la macchina si trova).

2. $\forall t,\ \forall i,\ \exists! j : C(i,j,t) = 1$
   (Ad ogni istante, in ogni singola cella, c'è un solo simbolo).

3. $\forall t,\ \exists! i : L(i,t) = 1$
   (Ad ogni istante, la testina legge un'unica cella).

4. Al tempo $t = 0$, imposto le variabili in modo che descrivano la configurazione iniziale di M:



5. Sia $F \subseteq Q$ l'insieme degli stati finali.
   $\exists u : q_u \in F$ e $S(u, p(n)) = 1$
   (La macchina deve terminare in uno stato di accettazione).

6. $\forall t, \forall i, \quad L(i,t) = 0 \implies C(i,j,t) = C(i,j,t+1)$
   (Il contenuto di una cella non cambia all'istante successivo se la testina non vi si trovava sopra).

7. $\forall t, \forall i, \quad L(i,t) = 1 \implies$
   Le variabili $S(u,t+1), C(i,j,t+1), L(i,t+1)$ assumono valori compatibili con le transizioni della funzione δ di M.

Traduzione in Polinomi Booleani (Costruzione di $f(w)$)

Definiamo una macro-funzione booleana U (Unicità) che garantisce che **una e una sola** variabile di un insieme sia vera:

$$U(y_1, y_2, \ldots, y_k) = y_1 \vee y_2 \vee \cdots \vee y_k = \left(\bigvee_{i=1}^{k} y_i \right) \wedge \bigwedge_{i \neq j} (y'_i \vee y'_j)$$

Esempio per $k = 3$:

$$U(y_1, y_2, y_3) = (y_1 \vee y_2 \vee y_3) \wedge (y'_1 \vee y'_2) \wedge (y'_1 \vee y'_3) \wedge (y'_2 \vee y'_3)$$

Osservazione: $U(y_1, \ldots, y_k) = 1 \iff \exists! i : y_i = 1$

Traduciamo ora ciascuna delle 7 condizioni in una corrispondente sotto-formula booleana:

1) Formula per l'unicità dello stato:

$$\bigwedge_{t=0}^{p(n)} U\Big(S(1,t), S(2,t), \ldots, S(s,t)\Big) = \mathbf{A}$$

2) Formula per l'unicità del simbolo in ogni cella:

$$\bigwedge_{t=0}^{p(n)} \bigwedge_{i=1}^{p(n)+1} U\Big(C(i,1,t), C(i,2,t), \ldots, C(i,r,t)\Big) = \mathbf{B}$$

3) Formula per l'unicità della posizione della testina:

$$\bigwedge_{t=0}^{p(n)} U\Big(L(1,t), L(2,t), \ldots, L(p(n)+1,t)\Big) = \mathbf{C}$$

4) Formula per l'inizializzazione ($t = 0$):

$$S(1,0) \wedge L(1,0) \wedge \bigwedge_{i=1}^{n} C(i, \text{“}w_i\text{”}, 0) \wedge \bigwedge_{i=n+1}^{p(n)+1} C(i, \text{“}*\text{”}, 0) = \mathbf{D}$$

5) Formula per l'accettazione finale:

$$\bigvee_{q_u \in F} S(u, p(n)) = \mathbf{E}$$

6) Formula per l'inerzia del nastro (dove la testina non c'è, niente cambia):

$$\bigwedge_{t=0}^{p(n)} \bigwedge_{i=1}^{p(n)+1} \left[L(i,t) \vee \bigwedge_{j=1}^{r} \Big(C(i,j,t) \text{ XNOR } C(i,j,t+1) \Big) \right] = \mathbf{F}$$

Nota esplicativa: Si ricorda che l'operatore di uguaglianza logica (XNOR) si esprime in CNF come $z(x,y) = (x \vee y') \wedge (x' \vee y)$.

7) Formula per la validità delle transizioni δ:

$$\mathbf{G}_{u,t,i,j} = S(u,t)' \vee L(i,t)' \vee C(i,j,t)' \vee \bigvee_{(\quad)\in\delta(q_u,a_j)} \Big(S(\quad,t+1) \wedge C(i,\quad,t+1) \wedge L(\quad,t+1)\Big)$$

Il polinomio totale è composto dalla congiunzione di tutti questi polinomi.
Osserviamo che tutti i polinomi sopra sono in CNF, tranne quello relativo alla proprietà 7, che però può essere trasformato in CNF in tempo polinomiale. ■

3.6 Proprietà della classe NP-Completo e NP-Difficile

Proposizione: Supponiamo che $\mathbf{P} = \mathbf{NP}$. Sia $L \in \mathcal{NP}$, con $L \neq \emptyset$ e $\overline{L} \neq \emptyset$ (quindi L è un linguaggio né vuoto né totale). Allora $L \in \mathbf{NPC}$ (cioè L è NP-Completo).

Dimostrazione:
Sappiamo che $L \in \mathcal{NP}$. Facciamo vedere che L è NP-difficile.
Sia $Q \in \mathcal{NP}$, descriviamo una riduzione polinomiale da Q a L. Poiché stiamo supponendo $\mathbf{P} = \mathbf{NP}$, abbiamo che $Q \in \mathcal{P}$.

Poiché L non è banale, sappiamo per ipotesi che:

- $\exists \alpha \in L$
- $\exists \beta \notin L$

Costruiamo la funzione di riduzione f:

$$f(w) = \begin{cases} \alpha & \text{se } w \in Q \\ \beta & \text{se } w \notin Q \end{cases}$$

Dato che $Q \in \mathcal{P}$, per calcolare f e decidere se $w \in Q$ mi basta usare una Macchina di Turing deterministica polinomiale. Quindi f è calcolabile in tempo polinomiale.

Osservazione:

- $\emptyset$ **non** è NP-difficile. Se prendiamo un generico $Q \in \mathcal{NP}$, esiste una riduzione polinomiale da Q a $\emptyset$? **NO!** Per evidenti ragioni insiemistiche, non avremmo alcun elemento in $\emptyset$ su cui mappare le stringhe accettate di Q.
- Σ^* **non** è NP-difficile. Il motivo è analogo al precedente (non avremmo elementi al di fuori di Σ^* su cui mappare le stringhe rifiutate di Q).

Proposizione (Transitività della NP-difficoltà): Se L è NP-difficile e f è una riduzione polinomiale da L a Q ($L \leq_p Q$), allora Q è NP-difficile.

Dimostrazione:
Dato un generico $R \in \mathcal{NP}$, descriviamo una riduzione polinomiale da R a Q.

1. Poiché L è NP-difficile e $R \in \mathcal{NP}$, $\exists g$ riduzione polinomiale da R a L.
   Ovvero $g : \Sigma_R^* \to \Sigma_L^*$ tale che $w \in R \iff g(w) \in L$.
2. Sappiamo per ipotesi che L è polinomialmente riducibile a Q.
   Ovvero $\exists f : \Sigma_L^* \to \Sigma_Q^*$ tale che $w \in L \iff f(w) \in Q$.

Osserviamo che la composizione $f \circ g : \Sigma_R^* \to \Sigma_Q^*$ garantisce che:

$$w \in R \iff f(g(w)) \in Q$$

Inoltre, essendo composizione di funzioni calcolabili in tempo polinomiale, $f \circ g$ è anch'essa calcolabile in tempo polinomiale. Questo dimostra che $R \leq_p Q$, quindi Q è NP-difficile.

3.7 Il Problema 3-SAT

Definizione (3-SAT): Dato p polinomio booleano in 3-CNF, determinare se p è soddisfacibile.

In 3-CNF, ogni clausola contiene esattamente 3 letterali.

Osservazione: 3-SAT $\in \mathcal{NP}$.
Facciamo vedere che 3-SAT è NP-difficile. Cerchiamo una riduzione polinomiale da SAT a 3-SAT.
Sia p un polinomio booleano in CNF: $p = u_1 \wedge u_2 \wedge \cdots \wedge u_m$ (dove u_i sono le clausole).
Analizziamo una singola clausola u di p e vediamo come trasformarla in una clausola $\tilde{u}$ equivalente per 3-SAT:

1. **Se $u = v$ (un solo letterale):**
   $\tilde{u} = (v \vee x \vee y) \wedge (v \vee x' \vee y) \wedge (v \vee x \vee y') \wedge (v \vee x' \vee y')$
   (dove x, y sono variabili nuove). Si ha che u è soddisfacibile $\iff$ $\tilde{u}$ è soddisfacibile.

2. **Se $u = v_1 \vee v_2$ (due letterali):**
   $\tilde{u} = (v_1 \vee v_2 \vee x) \wedge (v_1 \vee v_2 \vee x')$
   (dove x è una variabile nuova).

3. **Se $u = v_1 \vee v_2 \vee v_3$ (tre letterali):**
   $\tilde{u} = u$ (la clausola è già nel formato corretto).

4. **Se $u = v_1 \vee v_2 \vee \cdots \vee v_n$ con $n \geq 4$:**
   Introduciamo $n - 3$ nuove variabili $y_1, \ldots, y_{n-3}$ e trasformiamo:

$$\tilde{u} = (v_1 \vee v_2 \vee y_1) \wedge (v_3 \vee y_1' \vee y_2) \wedge \cdots \wedge (v_j \vee y_{j-2}' \vee y_{j-1}) \wedge \cdots \wedge (v_{n-1} \vee v_n \vee y_{n-3}')$$

Dimostrazione di validità per il caso 4 (u soddisfacibile $\iff$ $\tilde{u}$ soddisfacibile):
$\implies$ **Se u è soddisfacibile allora $\tilde{u}$ è soddisfacibile.**
Sia V l'insieme delle variabili in p. Sia $t : V \to \{0, 1\}$ un assegnamento tale che $t(u) = 1$.
Sia v_j il **primo** letterale di u tale per cui $t(v_j) = 1$.
Definiamo un nuovo assegnamento esteso $\tilde{t} : V \cup \{y_1, \ldots, y_{n-3}\} \to \{0, 1\}$ come segue:

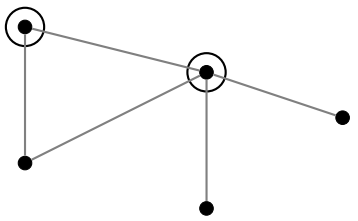
$$\tilde{t}(x) = \begin{cases} t(x) & \text{se } x \in V \\ 1 & \text{se } x \in \{y_1, \ldots, y_{j-2}\} \\ 0 & \text{se } x \in \{y_{j-1}, \ldots, y_{n-3}\} \end{cases}$$

Tale assegnamento rende vere tutte le nuove clausole.
$\impliedby$ **Se $\tilde{u}$ è soddisfacibile allora u è soddisfacibile.**
Sia $\tilde{t} : V \cup \{y_1, \ldots, y_{n-3}\} \to \{0, 1\}$ un assegnamento tale che $\tilde{t}(\tilde{u}) = 1$.
Sia $t = \tilde{t}|_V$ (la restrizione di $\tilde{t}$ alle sole variabili originali di p). Si dimostra che $t(u) = 1$.

3.8 Problema del Vertex Cover (VC)

Sia $G = (V, E)$ un grafo non orientato.

Definizione: Un **Vertex Cover** in G è un sottoinsieme di vertici $C \subseteq V$ tale che, $\forall \{x, y\} \in E$, si ha $x \in C$ oppure $y \in C$ (ogni arco è "coperto" da almeno uno dei suoi vertici estremi).



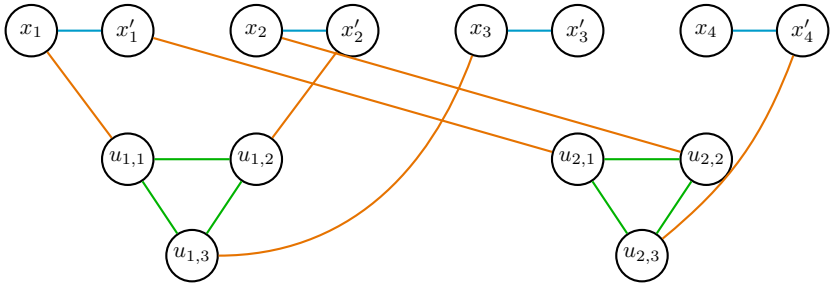
I due vertici cerchiati formano un Vertex Cover perché toccano tutti gli archi.

Definizione del Problema (VC): Dato un grafo non orientato $G = (V, E)$ e un intero positivo K, determinare se G possiede un Vertex Cover di cardinalità K.

Osservazione: VC $\in \mathcal{NP}$.
Costruiamo una riduzione polinomiale da 3-SAT a VC per dimostrare che VC è NP-difficile.
Sia p un polinomio booleano in 3-CNF composto da m clausole e n variabili $\{x_1, \ldots, x_n\}$:

$$p = (u_{1,1} \vee u_{1,2} \vee u_{1,3}) \wedge (u_{2,1} \vee u_{2,2} \vee u_{2,3}) \wedge \cdots \wedge (u_{m,1} \vee u_{m,2} \vee u_{m,3})$$

Esempio di costruzione: Sia $p = (x_1 \vee x_2' \vee x_3) \wedge (x_1' \vee x_2 \vee x_4')$.
Costruiamo il grafo corrispondente $G(p)$ con la seguente logica:



Osservazione: In un Vertex Cover di $G(p)$ ci devono essere almeno $n + 2m$ vertici.

- n vertici per coprire gli archi orizzontali in alto (dobbiamo scegliere per forza uno tra x_i e x_i').
- $2m$ vertici per coprire i triangoli (per coprire un triangolo interamente servono almeno 2 dei suoi vertici).

Facciamo vedere che p è soddisfacibile $\iff$ $G(p)$ ha un Vertex Cover di cardinalità $K(p) = n + 2m$.
$\implies$ Sia t un assegnamento tale che $t(p) = 1$.

- Per ogni $i = 1 \ldots n$, scegliamo il nodo x_i se $t(x_i) = 1$, oppure il nodo x_i' se $t(x_i) = 0$.
- Per ogni clausola, individuiamo un letterale che la soddisfa (essendo $t(p) = 1$, ce n'è almeno uno) e scegliamo i rimanenti 2 nodi del triangolo da mettere nel V.C.

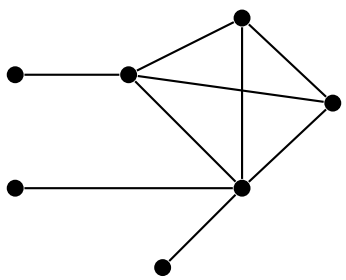
L'insieme dei vertici scelti ha cardinalità esattamente $n + 2m$ ed è un Vertex Cover valido per $G(p)$.
$\impliedby$ Sia C un Vertex Cover valido di cardinalità $n + 2m$. Poiché per coprire i "baffi" orizzontali in alto abbiamo scelto esattamente un nodo tra ogni coppia (x_i, x_i'), definiamo l'assegnamento t:

$$t(x_i) = \begin{cases} 1 & \text{se } x_i \in C \\ 0 & \text{se } x_i' \in C \end{cases}$$

Tale assegnamento t garantisce che p sia soddisfatto.

3.9 Problema della CLIQUE

Definizione (CLIQUE): Dato un grafo non orientato G e un intero K, determinare se esiste un sottografo completo di G avente K vertici.



Osservazione: CLIQUE $\in \mathcal{NP}$.

Proposizione: CLIQUE è NP-difficile.

Dimostrazione:
Descriviamo una riduzione polinomiale da 3-SAT a CLIQUE.
Sia P un polinomio booleano in 3-CNF:

$$P = u_1 \wedge u_2 \wedge \cdots \wedge u_K$$

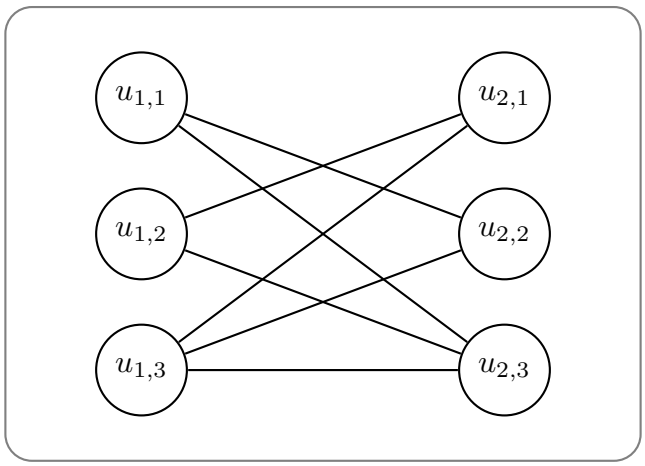
$$u_i = (u_{i,1} \vee u_{i,2} \vee u_{i,3})$$

Esempio di riduzione:
Consideriamo il seguente polinomio con un possibile assegnamento ($x_1 \to 1$, $x_2 \to 0$, $x_3 \to 1$, $x_4 \to 0$):

$$(\boxed{x_1} \vee x_2' \vee x_3) \wedge (x_1' \vee x_2 \vee \boxed{x_4'}) \wedge (x_2 \vee \boxed{x_3} \vee x_4')$$

Costruiamo il grafo corrispondente $G(P) = (V, E)$ secondo le seguenti regole:

- V: un vertice per ciascun letterale di ogni clausola.
- E: un lato (arco) tra ogni coppia di vertici, **tranne**:
  1. Vertici della stessa clausola.
  2. Vertici che rappresentano letterali opposti (es. x_1 e x_1').



Si dimostra che:
P **è soddisfacibile** $\iff$ $G(P)$ **ha un sottografo completo di cardinalità** K**.**
$\implies$ Sia t un assegnamento tale che $t(P) = 1$.
Per ogni clausola di P, scegliamo un letterale soddisfatto da t, e consideriamo il sottografo di $G(P)$ fondato dai vertici che rappresentano tali letterali.
$\impliedby$ Sia $\{v_1, \ldots, v_K\}$ un sottografo completo di $G(P)$.
$\forall i = 1, \ldots, K$, sia $u^{(i)}$ il letterale associato a v_i. Definiamo un assegnamento t tale che $t(u^{(i)}) = 1$.

- t è ben definito (perché nel sottografo completo non ci sono archi tra letterali opposti, quindi non assegneremo mai 1 sia a x che a x').
- t soddisfa P (perché abbiamo scelto esattamente un letterale per ogni clausola).

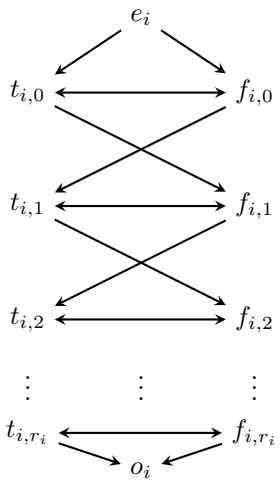
3.10 Problema del Circuito Hamiltoniano (HAM)

Proposizione: HAM è NP-difficile.

Dimostrazione:
Descriviamo una riduzione da 3-SAT a HAM.

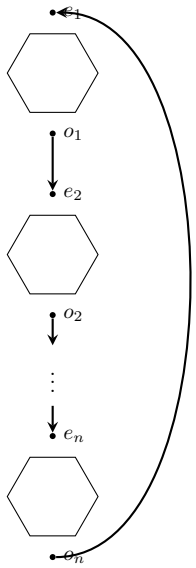
Sia $P = u_1 \wedge u_2 \wedge \cdots \wedge u_m$, dove le clausole sono $u_i = (u_{i,1} \vee u_{i,2} \vee u_{i,3})$.
Sia $V = \{x_1, \ldots, x_n\}$ l'insieme delle variabili.

Per ogni variabile x_i, costruiamo il seguente "gadget":



Quanti cammini hamiltoniani ci sono in questo singolo gadget (da e_i a o_i)? $\boxed{\mathbf{2}}$
(Uno percorrendo la colonna sinistra-destra e l'altro destra-sinistra).

Colleghiamo in serie tutti i gadget delle variabili:



Quanti circuiti hamiltoniani ci sono nel grafo totale senza clausole? $\boxed{2^n}$

Definiamo r_i = max tra le occorrenze di x_i e x'_i in P.

3.10.1 Costruzione del Grafo HAM Completo (Esempio)

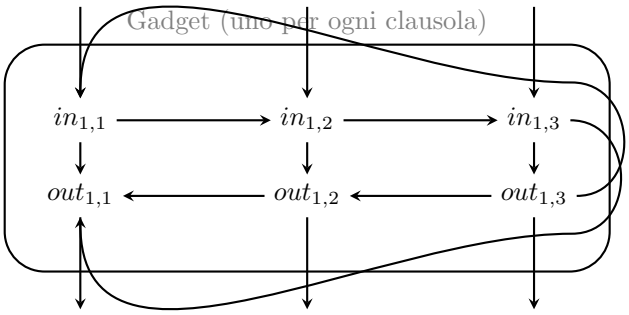
Esempio:

$$p = (x_1 \vee x_2 \vee x'_3) \wedge (x'_1 \vee x_2 \vee x'_4) \wedge (x_1 \vee x'_2 \vee x_4) \wedge (x'_1 \vee x_3 \vee x_4)$$

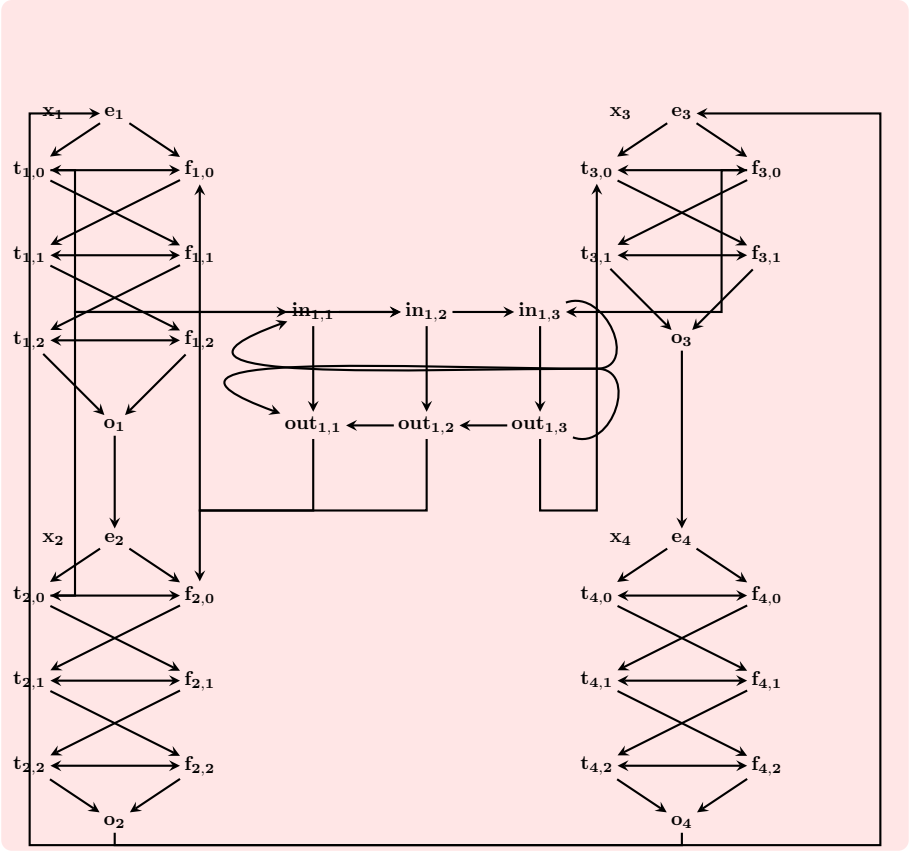
Devo sapere quanti nodi devo scrivere, cioè quanto vale r_i (che è il massimo tra le occorrenze di x_i e x'_i in p).

Esempio per x_1:
x_1 appare 2 volte, x_1' appare 2 volte $\implies r_1 = 2 \implies$ Sono 3 coppie di nodi centrali (indici 0, 1, 2).

Prima di vedere il grafo totale, definiamo il **gadget per la clausola**: ne inseriamo uno per ogni clausola e *ci entriamo solo se è vera la clausola.*

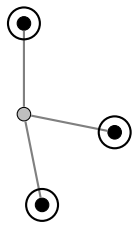


Di seguito è riportata la costruzione fedele del grafo completo per il polinomio dell'esempio (mostrando la connessione alla prima clausola $(x_1 \vee x_2 \vee x_3')$).



3.11 Problema dell'Insieme Indipendente (IS)

Definizione: Sia $G = (V, E)$ un grafo non orientato. Un sottoinsieme $I \subseteq V$ si dice **indipendente** quando $\forall x, y \in I$, si ha $\{x, y\} \notin E$.
(Ovvero, è un sottoinsieme di vertici senza lati che li collegano).



I tre vertici cerchiati
formano un Insieme Indipendente
perché non ci sono archi tra di loro.

Definizione del Problema (IS): Dato un grafo non orientato G e un intero K, determinare se G contiene un sottoinsieme indipendente di cardinalità K.

Riduzione polinomiale da CLIQUE a IS:
Mostriamo come trasformare un'istanza del problema CLIQUE in un'istanza del problema IS.

$$(G, K) \quad \longrightarrow \quad (\tilde{G}, K)$$

(Istanza del problema CLIQUE) *(Istanza per IS)*

Definiamo il nuovo grafo $\tilde{G} = (\tilde{V}, \tilde{E})$ dove:

- $\tilde{V} = V$ (i vertici rimangono esattamente gli stessi)
- $\tilde{E} =$ complementare di $E = \binom{V}{2} \setminus E$ (gli archi di $\tilde{G}$ sono tutti e soli gli archi **mancanti** in G)

Nota: la notazione $\binom{V}{2}$ indica l'insieme di tutte le possibili coppie di vertici, ovvero tutti i possibili archi di un grafo completo.

Per la costruzione stessa del grafo complementare, si deduce immediatamente l'equivalenza:

$$G \text{ ha un sottografo completo di cardinalità } K$$
$$\Updownarrow$$
$$\tilde{G} \text{ ha un sottoinsieme indipendente di cardinalità } K$$

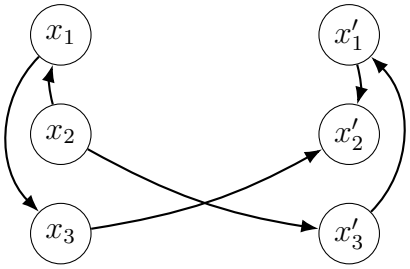
3.12 Problema 2-SAT

Definizione: Dato un polinomio booleano p in 2-CNF, determinare se p è soddisfacibile.
Sia $p = u_1 \wedge u_2 \wedge \cdots \wedge u_k$, dove ogni clausola è del tipo $u_i = v_{i,1} \vee v_{i,2}$.
Costruiamo un grafo orientato $G(p)$:

- **Vertici:** Per ogni variabile x che compare in p, inseriamo 2 vertici: x e x'.
- **Archi:** Per ogni clausola u_i, inseriamo 2 archi. Poiché $u_{i,1} \vee u_{i,2}$ equivale logicamente a $u'_{i,1} \to u_{i,2}$ e $u'_{i,2} \to u_{i,1}$, aggiungiamo proprio questi archi.

Esempio:

$$p = (x_1 \vee x'_2) \wedge (x'_1 \vee x_3) \wedge (x'_2 \vee x'_3)$$



Notazione: Siano α, β vertici. Scriviamo "$\alpha \rightsquigarrow \beta$" per significare "esiste un cammino da α a β".
Osservazione: Se $\alpha \rightsquigarrow \beta$, allora $\beta' \rightsquigarrow \alpha'$. Infatti, se esiste il cammino:

$$\alpha \to \gamma_1 \to \gamma_2 \to \gamma_3 \to \cdots \to \beta$$

Esisterà simmetricamente il cammino contrario negato:

$$\alpha' \leftarrow \gamma'_1 \leftarrow \gamma'_2 \leftarrow \gamma'_3 \leftarrow \cdots \leftarrow \beta'$$

Proposizione: p è soddisfacibile $\iff$ non esiste alcuna variabile x in p tale che in $G(p)$ si abbia contemporaneamente $x \rightsquigarrow x'$ e $x' \rightsquigarrow x$.

Dimostrazione ($\Longrightarrow$): Sia x tale che $x \rightsquigarrow x'$ e $x' \rightsquigarrow x$. Facciamo vedere che in questo caso p non è soddisfacibile. Sia t un generico assegnamento di verità.

1. Se $t(x) = 1$:
   Consideriamo il cammino $x \rightsquigarrow x'$:

   $$x \to \gamma_1 \to \cdots \to \alpha \to \beta \to \cdots \to \gamma_n \to x'$$

   Dato che partiamo da un valore vero ($t(x) = 1$) e arriviamo a un valore falso ($t(x') = 0$), ci deve essere per forza un arco intermedio $\alpha \to \beta$ in cui si passa da 1 a 0. Ovvero tale che $t(\alpha) = 1$ e $t(\beta) = 0$.
   L'arco $\alpha \to \beta$ deriva dalla clausola logica $\alpha' \vee \beta$. Ma se $t(\alpha) = 1$, allora $t(\alpha') = 0$. Essendo anche $t(\beta) = 0$, la clausola $\alpha' \vee \beta$ valuta a Falso.
   Quindi in p c'è la clausola $\alpha' \vee \beta$ che non è soddisfatta da $t \implies p$ non può essere soddisfatto.

2. Se $t(x) = 0$:
   Il ragionamento è del tutto analogo ma partendo dal cammino $x' \rightsquigarrow x$, portando allo stesso modo a un assurdo.

Dimostrazione ($\Longleftarrow$): Senza perdita di generalità, supponiamo che esista un letterale α tale che in p compaiono sia α che α' (altrimenti p sarebbe banalmente soddisfacibile).

Se, per assurdo, per tutti i letterali α tali che α, α' compaiono in p, si avesse $\alpha \rightsquigarrow \alpha'$, allora se β è uno di questi, avremmo $\beta \rightsquigarrow \beta'$ e contemporaneamente $\beta' \rightsquigarrow \beta$ (Assurdo, perché andrebbe contro l'ipotesi iniziale).

Pertanto esiste un letterale α di p tale che α' è anch'esso letterale di p e **non** c'è alcun cammino da α a α' ($\alpha \not\rightsquigarrow \alpha'$).

Sia α uno di tali letterali. Definiamo l'assegnamento t ponendo:

1. $t(\alpha) = 1$

2. $\forall \beta$ tale che $\alpha \rightsquigarrow \beta$, poniamo $t(\beta) = 1$

Osservazione: Verifichiamo che fin qui t è ben definito, perché non può accadere che debba porre $t(\beta') = 1$, in quanto $\alpha \not\rightsquigarrow \beta'$. Infatti, se per assurdo $\alpha \rightsquigarrow \beta'$ si avrebbe di conseguenza $\beta \rightsquigarrow \alpha'$. Inoltre, combinando i cammini: se $\alpha \rightsquigarrow \beta$ e $\beta \rightsquigarrow \alpha'$, si otterrebbe per transitività $\alpha \rightsquigarrow \alpha'$, che è un assurdo per come abbiamo scelto α.

$$\begin{tikzcd} & \beta \arrow[dr, "\rightsquigarrow"] & \\ \alpha \arrow[ur, "\rightsquigarrow"] \arrow[dr, dashed, red, "\rightsquigarrow"'] & & \alpha' \\ & \beta' & \end{tikzcd}$$

$\alpha \rightsquigarrow \alpha'$ **ASSURDO**

Algoritmo Deterministico Polinomiale per decidere se p è soddisfacibile
Per determinare se p è soddisfacibile: Dato p:

1. Costruisco $G(p)$.
2. Per ogni variabile x di p, controllo se ci sono i cammini $x \rightsquigarrow x'$ e $x' \rightsquigarrow x$.
3. Se ci sono entrambi per una qualche variabile, allora p non è soddisfacibile. Altrimenti, p è soddisfacibile.

Questo prova costruttivamente che **2-SAT** $\in \mathcal{P}$.

3.13 Linguaggi NP-Intermedi (NPI)

Se $\mathcal{P} \neq \mathcal{NP}$: Esiste una classe di linguaggi chiamati NP-Intermedi, definiti come:

$$NPI = \mathcal{NP} \setminus (\mathcal{P} \cup NPC)$$

Teorema (Ladner): Se $\mathcal{P} \neq \mathcal{NP} \implies NPI \neq \emptyset$ (cioè questi linguaggi intermedi esistono).

3.14 La classe co-NP

Sappiamo che $\mathcal{P}$ è chiuso rispetto alla complementazione.
Proposizione (domanda): Anche $\mathcal{NP}$ è chiusa per complementazione? Ovvero, $\mathcal{NP} \stackrel{?}{=} co-\mathcal{NP}$.
Definizione:

$$co-\mathcal{NP} = \{L \text{ linguaggio} \mid L' \in \mathcal{NP}\}$$

(Dove L' è il linguaggio complemento).
Esempio: UNSAT = Dato un polinomio booleano p, determinare se p **non** è soddisfacibile.

$$\text{UNSAT} \in co-\mathcal{NP}$$

Proposizione: $\mathcal{NP} \neq co-\mathcal{NP} \implies \mathcal{P} \neq \mathcal{NP}$.
Dimostrazione: Se per ipotesi $\mathcal{NP} \neq co-\mathcal{NP}$, sapendo già con certezza che $\mathcal{P} = co-\mathcal{P}$ (poiché la classe $\mathcal{P}$ è chiusa per complementazione), ne consegue logicamente che $\mathcal{P} \neq \mathcal{NP}$.

Proposizione: $L \in co-\mathcal{NP}$ e $L \in MPC \implies co-\mathcal{NP} = \mathcal{NP}$.
Dimostrazione: Dimostriamo le due inclusioni.
Parte 1: $co-\mathcal{NP} \subseteq \mathcal{NP}$
Sia $Q \in co-\mathcal{NP} \implies Q' \in \mathcal{NP}$. Poiché $L \in NPC$ (quindi NP-Completo), esiste una funzione f di riduzione polinomiale da Q' a L. Osserviamo che f è **anche** riduzione polinomiale da Q a $L' \in \mathcal{NP}$.
Possiamo allora definire un algoritmo polinomiale non deterministico per decidere Q: Data una stringa w:

1. Calcolo $f(w)$
2. Decido se $f(w) \in L'$

Poiché $L \in co-\mathcal{NP}$, sappiamo per definizione che $L' \in \mathcal{NP}$. Quindi l'algoritmo sopra appartiene a $\mathcal{NP}$. Di conseguenza, $Q \in \mathcal{NP}$. Abbiamo provato che $co-\mathcal{NP} \subseteq \mathcal{NP}$.

Parte 2: $\mathcal{NP} \subseteq co-\mathcal{NP}$

Sia $Q \in \mathcal{NP} \implies Q' \in co-\mathcal{NP}$ (per definizione di co-NP). Dalla dimostrazione precedente sappiamo che se $Q' \in co-\mathcal{NP} \implies Q' \in \mathcal{NP}$. Se il complemento Q' sta in $\mathcal{NP}$, allora per definizione $Q \in co-\mathcal{NP}$.

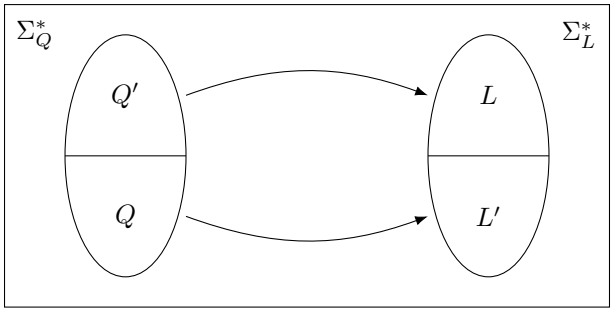
Unendo le due parti, otteniamo che $co-\mathcal{NP} = \mathcal{NP}$.

Osservazione:

$$\text{UNSAT} \in \mathcal{NP} \iff co-\mathcal{NP} = \mathcal{NP}$$

Spiegazione: Poiché UNSAT è un problema **co-NP-completo** (cioè tra i più difficili della sua classe), se dimostrassimo che appartiene a $\mathcal{NP}$, allora "trascinerebbe" con sé tutta la classe $co-\mathcal{NP}$, facendola collassare in un'uguaglianza con $\mathcal{NP}$.

Schema della Riduzione per Complementi: Il seguente diagramma mostra visivamente come si comportano le riduzioni con i linguaggi complemento (come visto nelle dimostrazioni precedenti): se una funzione mappa un linguaggio Q nel linguaggio L', essa mapperà in parallelo il complemento Q' nel complemento L (poiché il complemento del complemento torna all'originale).



3.14.1 Proprietà di Chiusura dei Linguaggi co-NP

Analizziamo alcune operazioni insiemistiche sui linguaggi $co-\mathcal{NP}$ attraverso dei pratici esercizi teorici sulle loro proprietà di chiusura.

Esercizio 1 (Intersezione): Dimostrare che se $L_1, L_2 \in co-\mathcal{NP}$, allora la loro intersezione $L_1 \cap L_2 \in co-\mathcal{NP}$.

Svolgimento: Per definizione di linguaggio complemento, sappiamo che $L_1', L_2' \in \mathcal{NP}$. Per le leggi di De Morgan, possiamo scrivere l'intersezione dei linguaggi originali come il complemento della loro unione:

$$L_1 \cap L_2 = (L_1' \cup L_2')'$$

Sappiamo che la classe $\mathcal{NP}$ è chiusa rispetto all'unione, quindi l'unione di due linguaggi in $\mathcal{NP}$ produce ancora un linguaggio in $\mathcal{NP}$:

$$\underbrace{L_1'}_{\in \mathcal{NP}} \cup \underbrace{L_2'}_{\in \mathcal{NP}} \in \mathcal{NP}$$

Di conseguenza, il complemento di questa unione apparterrà per definizione alla classe $co-\mathcal{NP}$. Pertanto, $(L_1' \cup L_2')' \in co-\mathcal{NP}$.

Esercizio 2 (Differenza): Assumendo che $L \in \mathcal{NP}$ e $L_1 \in co-\mathcal{NP}$ con l'ipotesi aggiuntiva che $L_1 \subseteq L$, dimostrare che:

$$L \setminus L_1 \in \mathcal{NP}$$

Svolgimento: Possiamo riscrivere l'operazione di differenza insiemistica come l'intersezione con il complemento del secondo insieme:

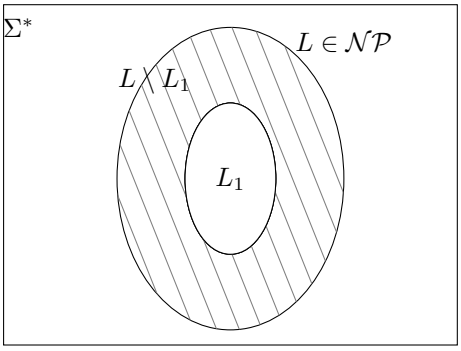
$$L \setminus L_1 = L \cap L_1'$$

Sappiamo dalle ipotesi che $L \in \mathcal{NP}$. Inoltre, poiché $L_1 \in co-\mathcal{NP}$, per definizione il suo complemento L_1' apparterrà alla classe $\mathcal{NP}$ (cioè $L_1' \in \mathcal{NP}$). Dato che l'intersezione di due linguaggi in $\mathcal{NP}$ è ancora un linguaggio in $\mathcal{NP}$ (poiché la classe $\mathcal{NP}$ è chiusa per l'intersezione), otteniamo:

$$\underbrace{L}_{\in \mathcal{NP}} \cap \underbrace{L_1'}_{\in \mathcal{NP}} \in \mathcal{NP}$$

Osservazione Grafica: Il seguente diagramma di Venn illustra visivamente i passaggi logici dell'Esercizio 2.

- Il rettangolo esterno Σ^* rappresenta l'universo (tutte le stringhe possibili per quel dato alfabeto).
- L'ellisse più grande rappresenta l'insieme L, che si trova in $\mathcal{NP}$.
- L'ellisse più piccola (bianca) al suo interno è l'insieme L_1, che è interamente contenuto in L ($L_1 \subseteq L$).
- L'area **tratteggiata** (simile a una ciambella) evidenzia il risultato dell'operazione $L \setminus L_1$. Prendendo tutti gli elementi dell'insieme L e "scartando" quelli che appartengono a L_1 (che algebricamente corrisponde proprio all'intersezione tra L e tutto ciò che sta *fuori* da L_1, ovvero L_1'), otteniamo la regione tratteggiata. Questo conferma anche visivamente perché la scomposizione matematica usata nello svolgimento è corretta.



3.15 Test di Primalità

Problema: Determinare se n è composto $\implies \exists d \mid n$, con $d \in [2, \sqrt{n}]$.

Dimostrazione: Se n è composto allora $n = m_1 \cdot m_2$. Per assurdo, se $m_1, m_2 > \sqrt{n}$ allora $n = m_1 \cdot m_2 > n$ **(Assurdo)**

Questo suggerisce il seguente algoritmo:
Algoritmo: Dato $n \in \mathbb{N}$, per ogni $d \in [2, \sqrt{n}]$ controllo se $d \mid n$:

- se sì $\implies$ n è composto
- se no $\implies$ n è primo

Per codificare l'input si usa la codifica binaria, dunque $\log n = l$ (lunghezza della codifica binaria di n), che diventa il parametro per calcolare la complessità.
La complessità dell'algoritmo è particolarmente grande: (almeno) $\Omega(\sqrt{n}) = \Omega(2^{l/2}) \implies$ esponenziale rispetto a l.
Vogliamo ridurne la complessità sfruttando proprietà note dei numeri primi:

Piccolo Teorema di Fermat:

1. p primo, $a \in \mathbb{N}$ allora $a^p \equiv a \pmod p$
2. p primo, $a \in \mathbb{N}$ coprimo con p (cioè $(a, p) = 1$), allora $a^{p-1} \equiv 1 \pmod p$

Algoritmo non deterministico:
Dato $n \in \mathbb{N}$ genero non deterministicamente $a \in \mathbb{N}$ tale che $(a, n) = 1$:

- se $a^{n-1} \equiv 1 \pmod{n}$ allora n è primo
- altrimenti n è composto.

Questo algoritmo **non è corretto**, perché utilizza il teorema di Fermat al "contrario", ma non vale il viceversa del teorema.

Esempio: $p = 341$, $a = 2$, quindi $2^{340} \equiv 1 \pmod{341}$ ma $341 = 11 \cdot 31$ (non è primo!).
Pur non essendo corretto ha una complessità migliore rispetto all'algoritmo precedente. Esiste infatti un algoritmo per il calcolo delle potenze modulari a^n (che calcola potenza e successione del modulo) che ha complessità $\mathcal{O}(\log^2 n) = \mathcal{O}(l^2)$, ovvero **polinomiale**. Riusciamo a "sistemare" il seguente algoritmo, mantenendolo polinomiale? Sì, ed è stato dimostrato nel 1975 grazie a questo teorema:

Teorema (Pratt): Sia $n \in \mathbb{N}$. Se $\exists a \in \mathbb{N}$ tale che $(a, n) = 1$, $a^{n-1} \equiv 1 \pmod{n}$, e $\forall q \mid n-1$ con q primo si ha che $a^{\frac{n-1}{q}} \not\equiv 1 \pmod{n}$, allora n è primo.
L'algoritmo basato su questo teorema ha complessità polinomiale.

Osservazione: Ogni numero naturale possiede meno fattori primi distinti rispetto ai caratteri nella sua rappresentazione binaria.

$$n = p_1^{\alpha_1} p_2^{\alpha_2} \dots p_r^{\alpha_r} \implies r \text{ fattori primi distinti}$$

Il caso peggiore si ha con $\alpha_1 = \alpha_2 = \dots = \alpha_r = 1$, dunque $n = p_1 \cdot p_2 \dots p_r > 2^r$.
Testare la condizione del piccolo teorema di Fermat per un certo numero di a tuttavia non funziona a causa della presenza dei *numeri di Carmichael*.

Definizione: Un numero $n \in \mathbb{N}$ si dice **numero di Carmichael** quando $\forall a \in \mathbb{N}$ con $(a, n) = 1$ vale $a^{n-1} \equiv 1 \pmod{n}$, ma n **non** è primo.

Esempio: 561 è un numero di Carmichael, infatti la proprietà vale per ogni a coprimo, ma $561 = 3 \cdot 11 \cdot 17$.

Teorema (Alford, Granville, Pomerance): Esistono infiniti numeri di Carmichael.

Idea: Si necessita quindi di una modifica al piccolo teorema di Fermat.

Teorema (Agrawal): Siano $n \geq 2 \in \mathbb{N}$ e $a \in \mathbb{N}$ con $(a, n) = 1$.

$$n \text{ è primo} \iff (x + a)^n \equiv x^n + a \pmod{n}$$

Dimostrazione: $\implies$) Supponiamo n primo: Sappiamo che $(x + a)^n = \sum_{k=0}^{n} \binom{n}{k} a^{n-k} x^k$ dove $\binom{n}{k} = \frac{n!}{k!(n-k)!} = \frac{n \cdot (n-1) \dots (n-k+1)}{k!}$. Se $0 < k < n$ allora $n \mid \binom{n}{k}$, dunque modulo n tutti i termini si annullano e rimangono solo i coefficienti con $k = 0$ e $k = n$. Dunque:

$$(x + a)^n = \sum_{k=0}^{n} \binom{n}{k} a^{n-k} x^k \equiv x^n + a^n \equiv x^n + a \pmod{n}$$

$\impliedby$) Supponiamo n composto. Dimostriamo che $\exists k$ con $0 < k < n$ tale che $n \nmid \binom{n}{k}$. Sfruttiamo il fatto che n è composto. Sia $p \mid n$ un suo fattore primo. Supponiamo $p^\alpha \mid n$ e $p^{\alpha+1} \nmid n$. Se prendiamo il coefficiente binomiale $\binom{n}{p}$ allora $n \nmid \binom{n}{p}$ perché:

$$\binom{n}{p} = \frac{n(n-1) \dots (n-p+1)}{p(p-1) \dots 2 \cdot 1}$$

Osserviamo che p^α divide il numeratore; inoltre, nei fattori da $(n-1) \dots (n-p+1)$ non compare nessun'altra occorrenza del fattore p, dunque $p^{\alpha+1} \nmid$ numeratore. Nel denominatore abbiamo un'unica occorrenza di p, dunque $p^\alpha \nmid \binom{n}{p}$, ma sapendo che $p^\alpha \mid n$, ne consegue che $n \nmid \binom{n}{p}$. Questo significa che la congruenza non vale se n è composto.

Il teorema di Fermat diventa così una condizione necessaria e sufficiente. Senza alcuna "precauzione", i coefficienti da testare sarebbero circa $n+1$, quindi dovremmo controllare $n = 2^l$ congruenze. Vediamo se è possibile ridurre i coefficienti.

Idea: Dividere i polinomi nella congruenza per $x^r - 1$ per un opportuno parametro r.

Algoritmo AKS: Agrawal, Kayal, Saxena (2002) hanno dimostrato che se n è composto e si sceglie un r giusto, allora è sufficiente testare la seguente congruenza "per pochi" a:

$$(x+a)^n \equiv x^n + a \pmod{n, x^r - 1}$$

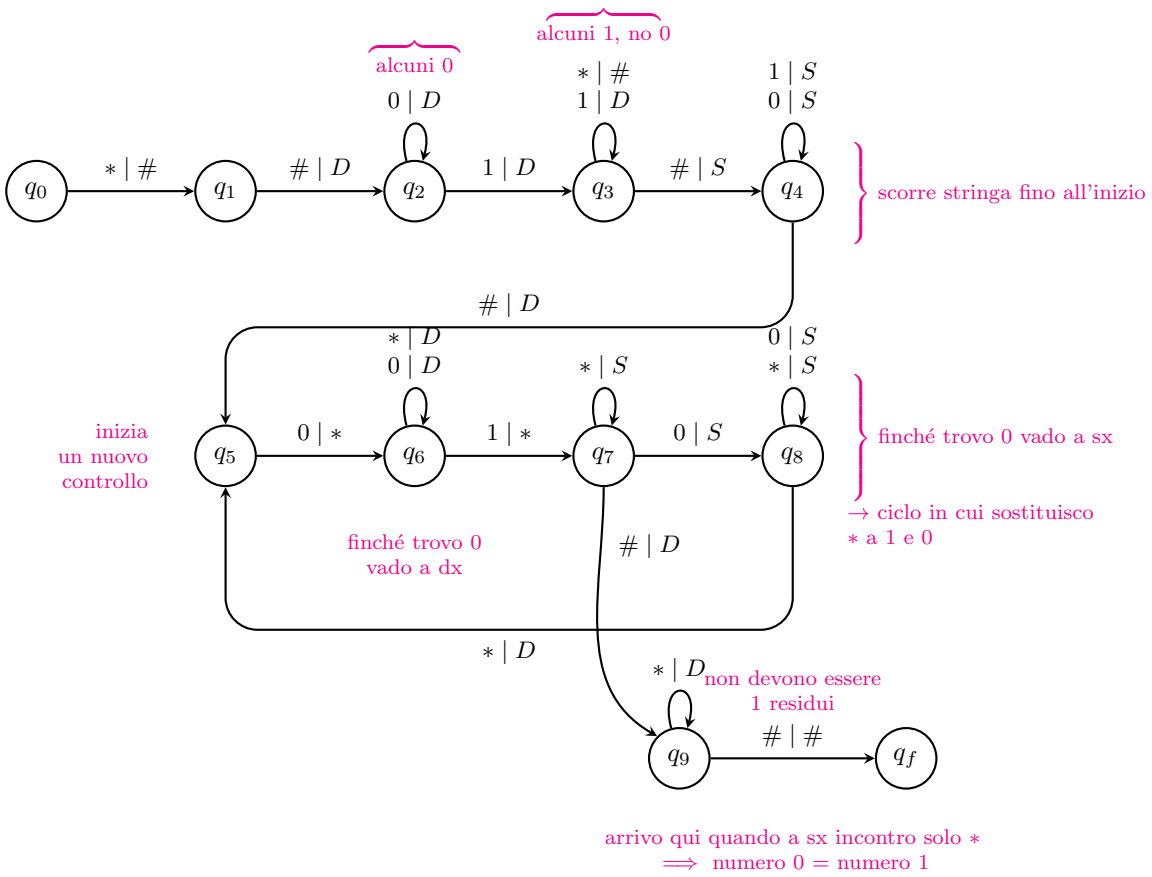
e se ne troverà sicuramente uno per cui non vale. Se invece tutte le congruenze valgono, allora il numero è primo.

L'efficienza dipende dalla scelta dei parametri:

- Necessario scegliere l'r giusto $\rightarrow$ ha lunghezza polinomiale rispetto all'input.
- Anche a ha una lunghezza polinomiale rispetto all'input.

L'algoritmo AKS è dunque **polinomiale**, e a differenza di quelli probabilistici, è anche **corretto e deterministico**.

3.16 Esercizio: Macchina di Turing e Transizioni



1) Linguaggio accettato: La macchina si aspetta stringhe del formato $\#000111\#\ldots$ Cioè stringhe del tipo $0^h 1^k$ con $h \geq 1, k \geq 1$. Per come cicla eliminando in coppia 0 e 1, accetta la stringa solo se in realtà $h = k$. Dunque il linguaggio accettato è:

$$L = \{0^h 1^h \mid h > 0\}$$

2) Studio del caso peggiore: Fissata una lunghezza della stringa pari a n, com'è fatta la stringa in cui la macchina esegue il numero massimo di transizioni? $\Longrightarrow$ Il caso peggiore si ha con una stringa **non accettata** ma che forza l'algoritmo a fare più passaggi possibili.

- Se n è **pari**: $0^{n/2}1^{n/2}$ (devo fare tutte le trasformazioni prima di finire)
- Se n è **dispari**: $0^{(n+1)/2}1^{(n-1)/2}$

Esempio Pratico: Tra 001 e 011, in quale viene eseguito il numero massimo di passi?

- per la stringa 001: 10 transizioni
- per la stringa 011: 8 transizioni

3) Calcolo della Complessità in Tempo: Il tempo calcolato $t_{TM}(n)$ dipenderà dal conteggio delle transizioni fatte avanti e indietro per fare "matching" tra zeri e uno.

$$t_{TM}(n) = 1 + 1 + \frac{n}{2} + \frac{n}{2} + 1 + 1 + n + 1 + \dots$$

$\Longrightarrow$ Per il calcolo esatto è necessario distinguere il caso in cui n sia pari dal caso in cui n sia dispari.